US012407533B1

(12) **United States Patent**
Stanislav Mario Georgiev et al.

(10) **Patent No.: US 12,407,533 B1**
(45) **Date of Patent: Sep. 2, 2025**

(54) **METHOD AND SYSTEM FOR NFT VALIDATION**

(71) Applicant: **Elazar Stanislav Mario Georgiev**, Jerusalem (IL)

(72) Inventors: **Elazar Stanislav Mario Georgiev**, Jerusalem (IL); **Leandro Morales Valle Cruz**, London (GB)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 464 days.

(21) Appl. No.: **17/685,893**

(22) Filed: **Mar. 3, 2022**

(51) **Int. Cl.**
H04L 9/00 (2022.01)
H04L 9/32 (2006.01)

(52) **U.S. Cl.**
CPC . ***H04L 9/50*** (2022.05); ***H04L 9/32*** (2013.01)

(58) **Field of Classification Search**
CPC H04L 9/50; H04L 9/32; H04L 9/0643
USPC 713/168
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

11,954,676 B1 * 4/2024 Richter G06Q 40/04
2007/0217676 A1 * 9/2007 Grauman G06V 10/764 382/190
2020/0285836 A1 * 9/2020 Davis H04L 63/0815
2021/0271508 A1 * 9/2021 Lee G06Q 20/405
2022/0069996 A1 * 3/2022 Xue H04L 9/3297
2023/0214819 A1 * 7/2023 Tham G06Q 20/3674 705/64
2023/0245137 A1 * 8/2023 McCullough G06Q 30/0185 705/318
2024/0378063 A1 * 11/2024 Steele, III G06F 9/451

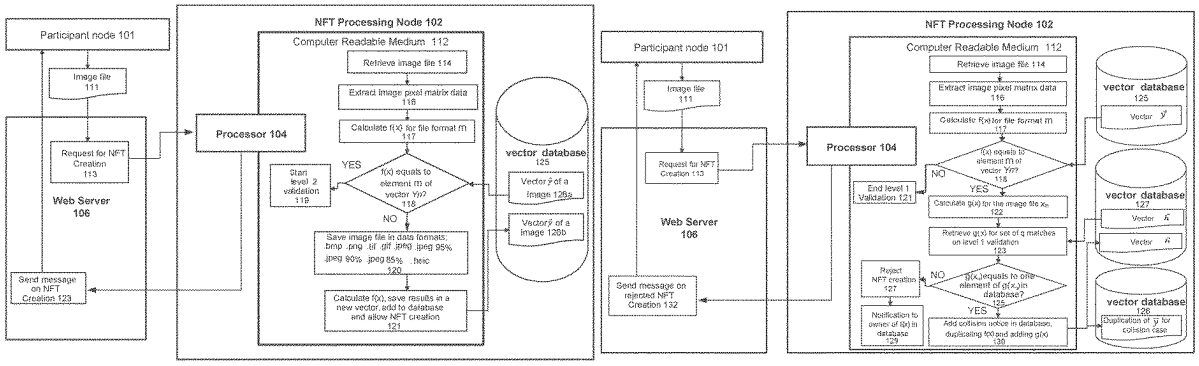
* cited by examiner

Primary Examiner — Clayton R Williams
(74) *Attorney, Agent, or Firm* — Michael D. Eisenberg

(57) **ABSTRACT**

An example NFT validation operation may include one or more of receiving an NFT creation request by an NFT processing node connected to a peer-to-peer file storage protocol system node over a blockchain; scanning, by the NFT processing node, the blockchain for an image file reference based on the request; retrieving, by the NFT processing node, the image file from the peer-to-peer file storage protocol system based on the image file reference; extracting, by the NFT processing node, image pixel matrix data from the image file; converting the image pixel matrix data into a plurality of image formats data; calculating a plurality of values of a bijective or hash function of each of the plurality of image formats data; and validating the NFT by comparing the plurality of the values of the bijective or hash function against pre-stored in a validation database value vectors comprising the plurality of values of the bijective or hash function of existing NFT images.

16 Claims, 15 Drawing Sheets



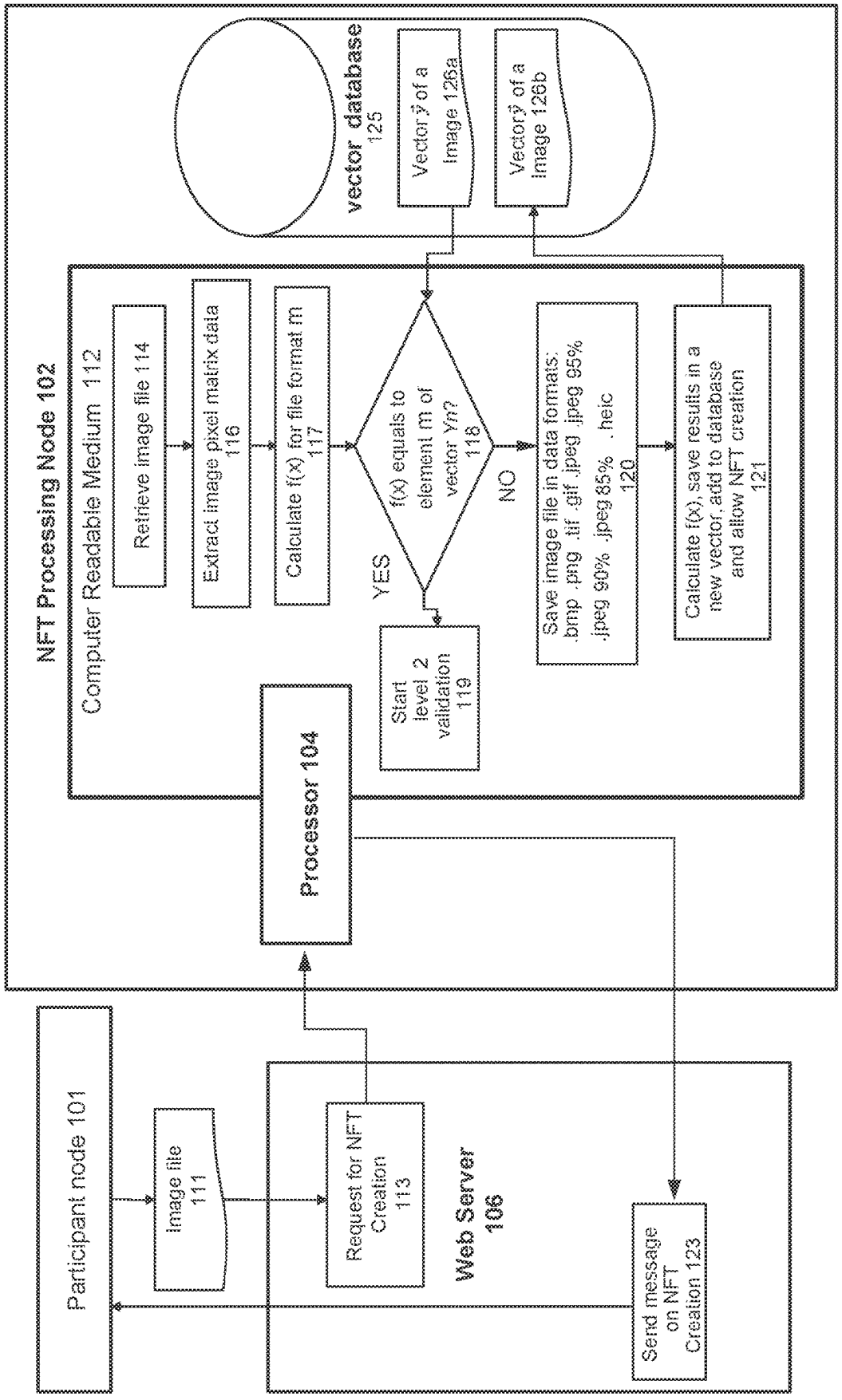

Participant node 101
Image file 111
Request for NFT Creation 113
Web Server 108
Send message on NFT Creation 123
NFT Processing Node 102
Computer Readable Medium 112
Processor 104
vector database 125
Vector $\vec{y}$ of a image 126a
Vector $\vec{y}$ of a image 126b
Retrieve image file 114
Extract image pixel matrix data 116
Calculate f(x) for file format m 117
f(x) equals to element m of vector Yn? 118
YES
NO
Start level 2 validation 119
Save image file in data formats: .bmp .png .tif .gif .jpeg .jpeg 95% .jpeg 90% .jpeg 85% .heic 120
Calculate f(x), save results in a new vector, add to database and allow NFT creation 121

FIG. 1A

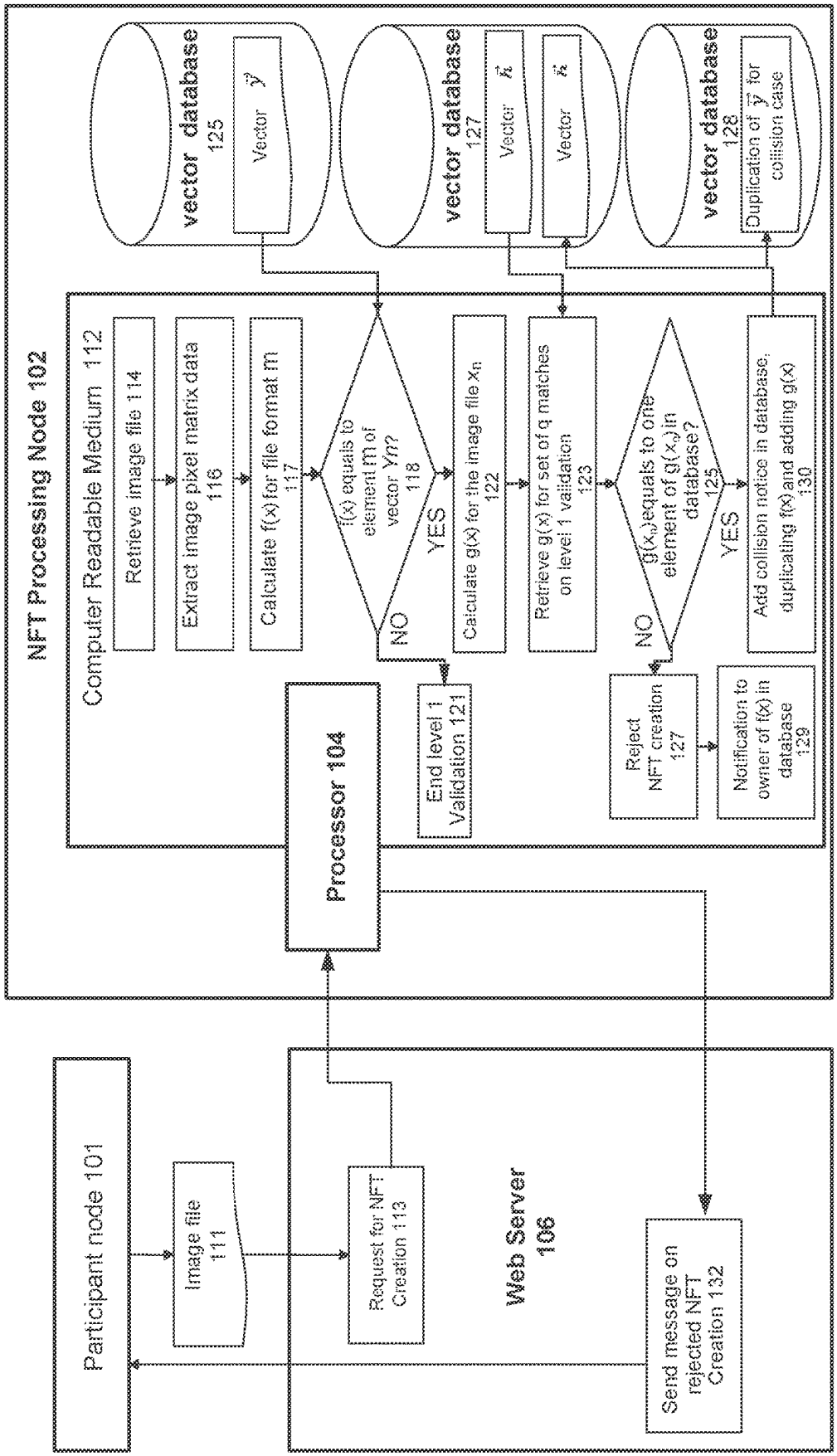

NFT Processing Node 102
Computer Readable Medium 112
Retrieve image file 114
Extract image pixel matrix data 116
Calculate f(x) for file format m 117
f(x) equals to element m of vector Ym? 118
YES
NO
Caiculate g(x) for the image file x_n 122
Retrieve g(x) for set of q matches on level 1 validation 123
$g(x_n)$ equals to one element of $g(x_q)$ in database? 125
YES
NO
Add collision notice in database, duplicating f(x) and adding g(x) 130
vector database 125
Vector $\vec{y}$
vector database 127
Vector $\vec{k}$
Vector $\vec{k}$
vector database 128
Duplication of $\vec{y}$ for collision case
Processor 104
End level 1 Validation 121
Reject NFT creation 127
Notification to owner of f(x) in database 129
Participant node 101
Image file 111
Request for NFT Creation 113
Web Server 106
Send message on rejected NFT Creation 132

FIG. 1B

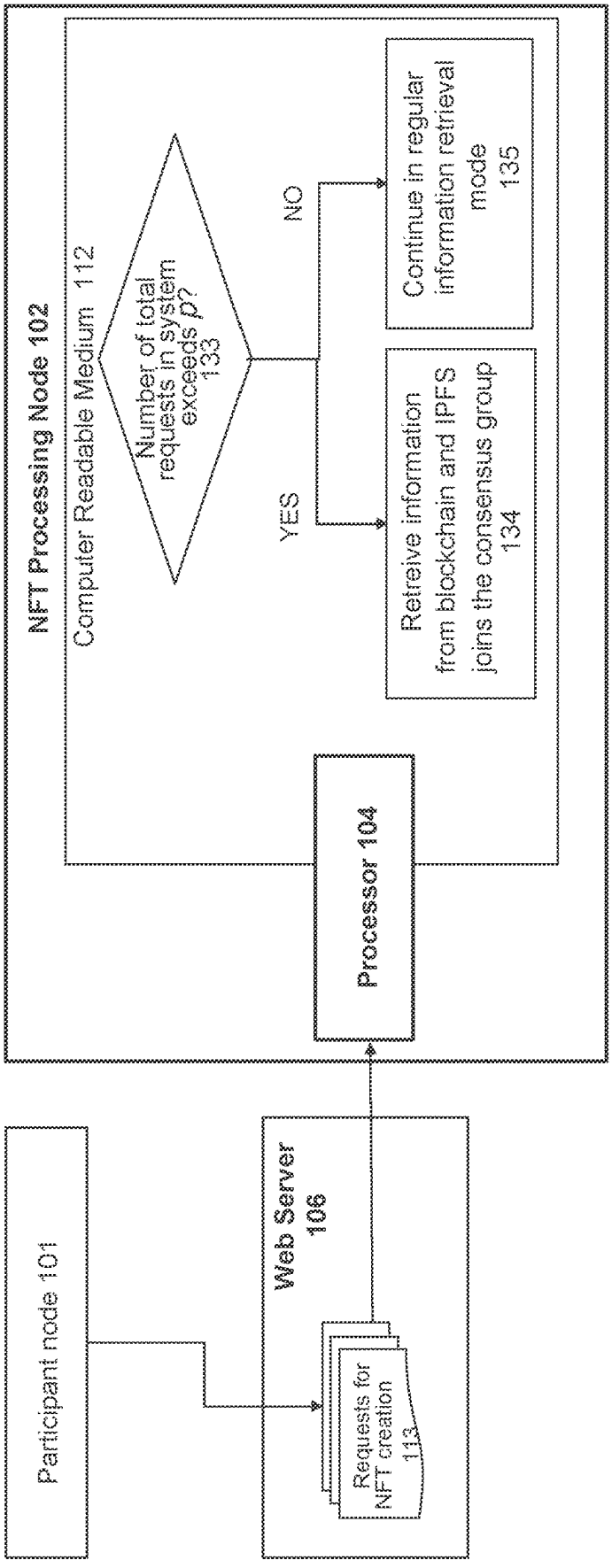

NFT Processing Node 102
Computer Readable Medium 112
Number of total requests in system exceeds D? 133
NO
YES
Continue in regular information retrieval mode 135
Retreive information from blockchain and IPFS joins the consensus group 134
Processor 104
Participant node 101
Web Server 106
Requests for NFT creation 113

FIG. 1C

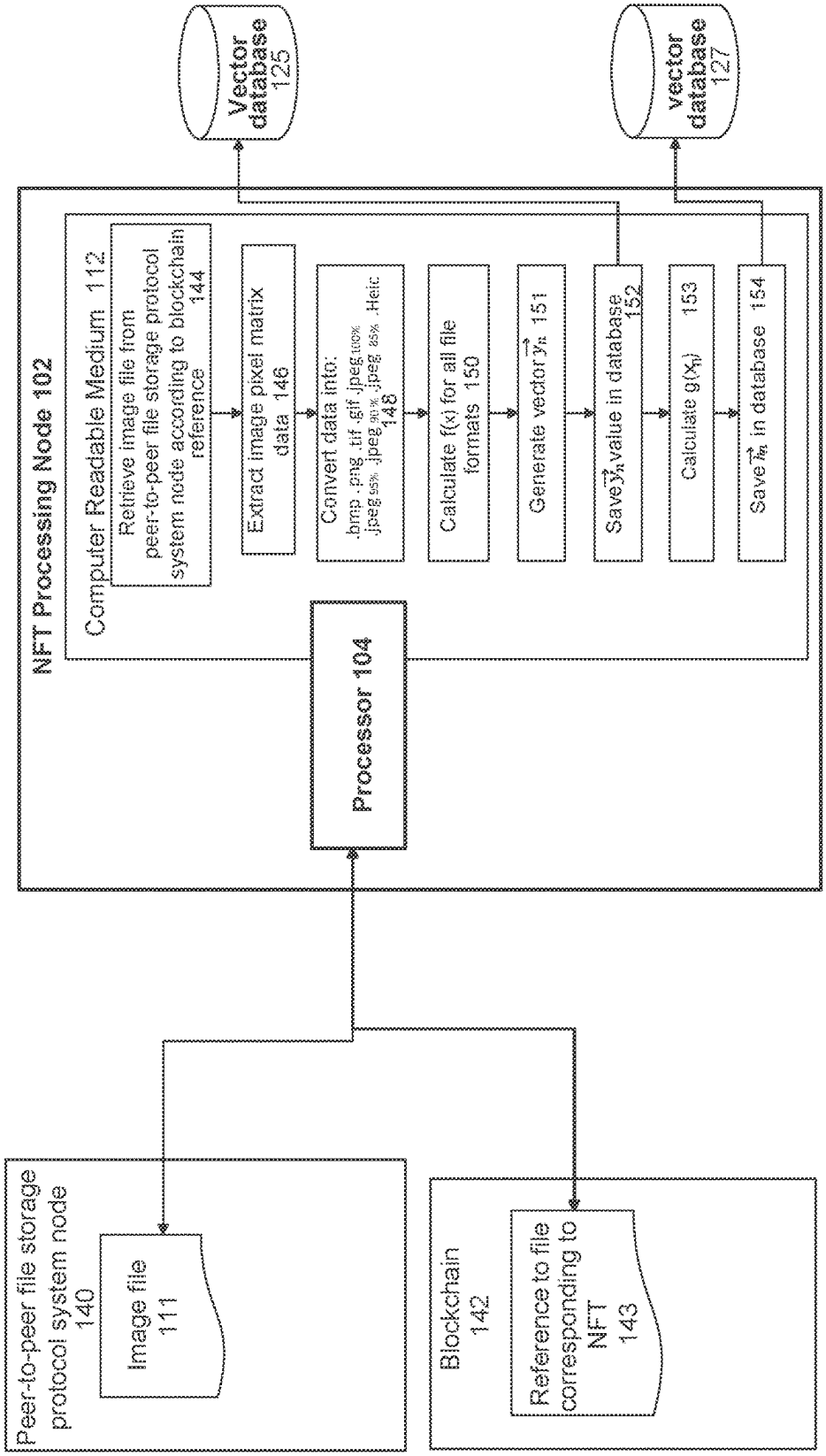

Vector database 125
vector database 127
NFT Processing Node 102
Computer Readable Medium 112
Retrieve image file from peer-to-peer file storage protocol system node according to blockchain reference 144
Extract image pixel matrix data 146
Convert data into: .bmp .png .tif .gif .jpeg100% .jpeg95% .jpeg90% .jpeg85% .Heic 148
Calculate f(x) for all file formats 150
Generate vector $\vec{y}_n$ 151
Save $\vec{y}_n$ value in database 152
Calculate $g(\vec{x}_n)$ 153
Save $\vec{w}_n$ in database 154
Processor 104
Peer-to-peer file storage protocol system node 140
Image file 111
Blockchain 142
Reference to file corresponding to NFT 143

FIG. 1D

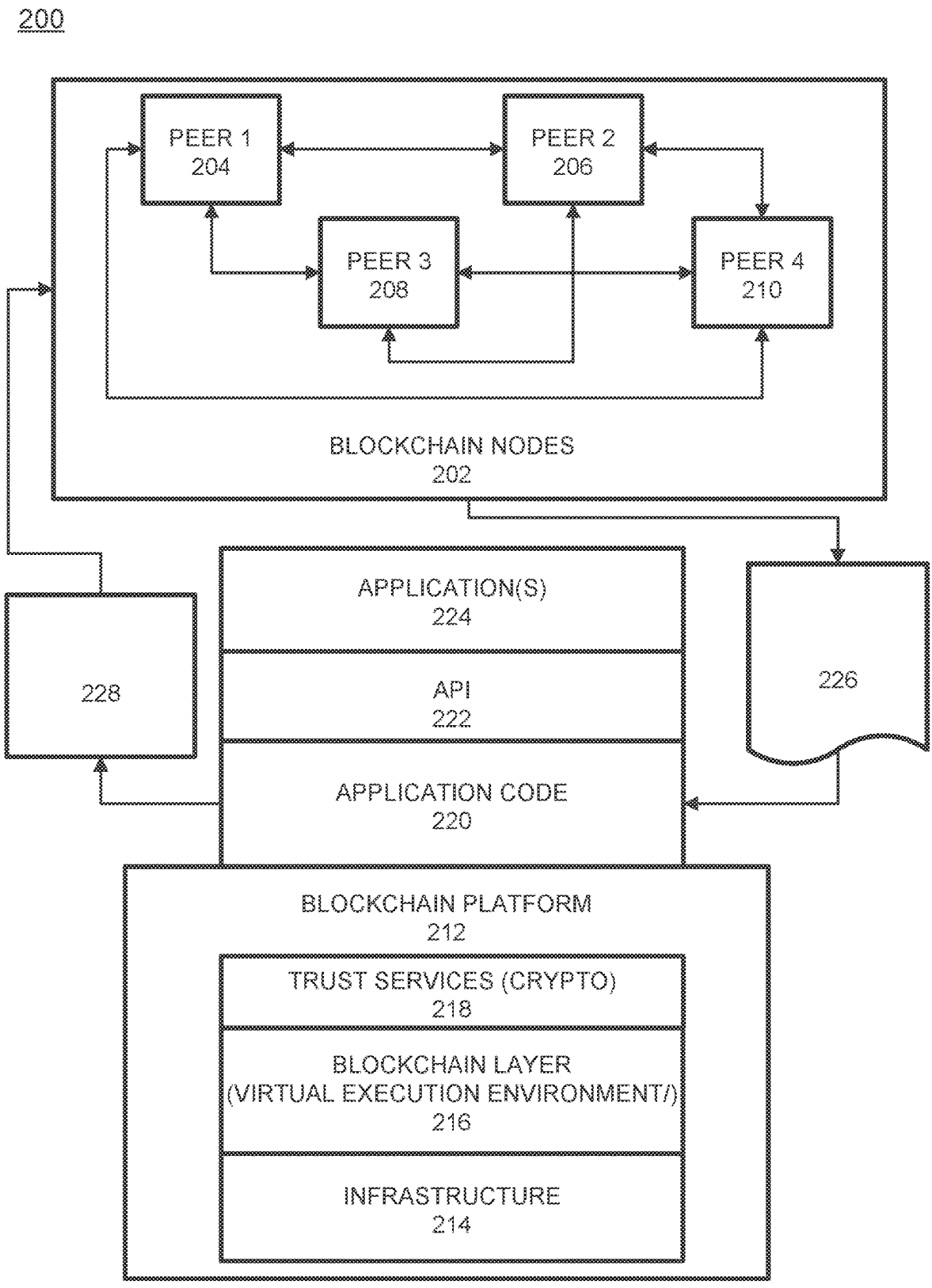

200
PEER 1
204
PEER 2
206
PEER 3
208
PEER 4
210
BLOCKCHAIN NODES
202
APPLICATION(S)
224
API
222
APPLICATION CODE
220
228
226
BLOCKCHAIN PLATFORM
212
TRUST SERVICES (CRYPTO)
218
BLOCKCHAIN LAYER
(VIRTUAL EXECUTION ENVIRONMENT/)
216
INFRASTRUCTURE
214

FIG. 2

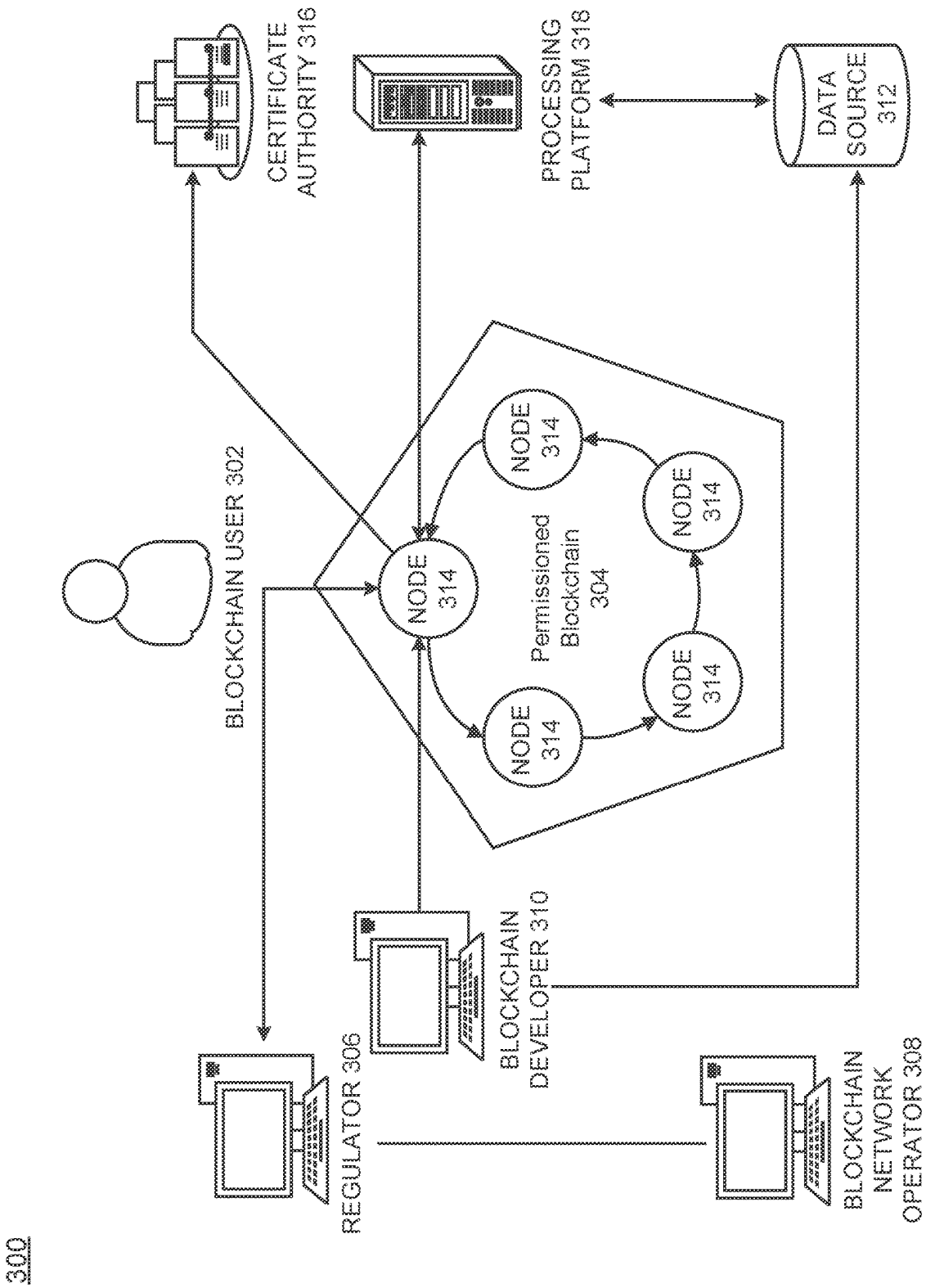

300
CERTIFICATE AUTHORITY 316
PROCESSING PLATFORM 318
DATA SOURCE 312
BLOCKCHAIN USER 302
NODE 314
NODE 314
NODE 314
NODE 314
NODE 314
Permissioned Blockchain 304
REGULATOR 306
BLOCKCHAIN DEVELOPER 310
BLOCKCHAIN NETWORK OPERATOR 308

FIG. 3A

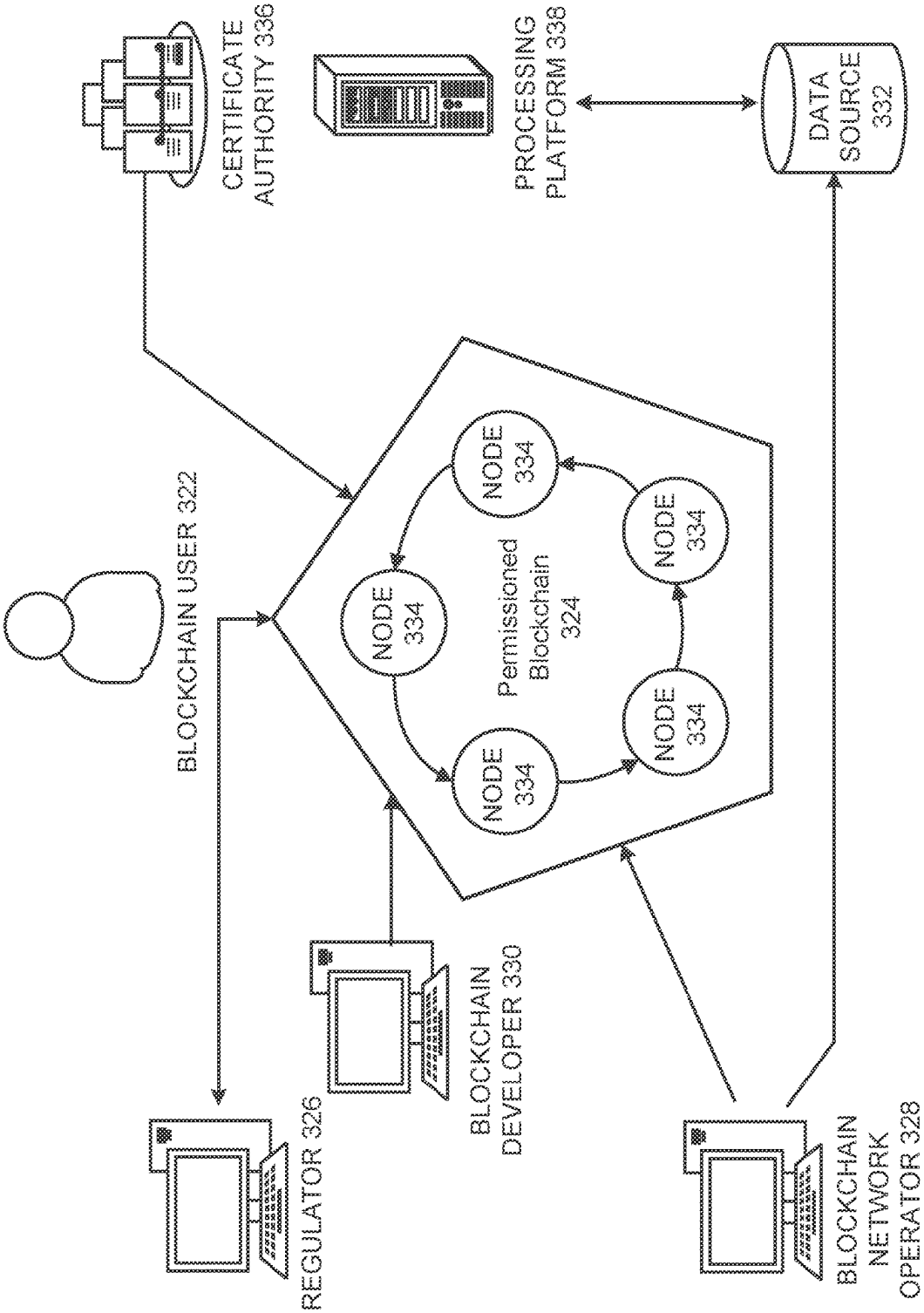

320
CERTIFICATE AUTHORITY 336
PROCESSING PLATFORM 338
DATA SOURCE 332
BLOCKCHAIN USER 322
NODE 334
NODE 334
NODE 334
NODE 334
Permissioned Blockchain 324
REGULATOR 326
BLOCKCHAIN DEVELOPER 330
BLOCKCHAIN NETWORK OPERATOR 328

FIG. 3B

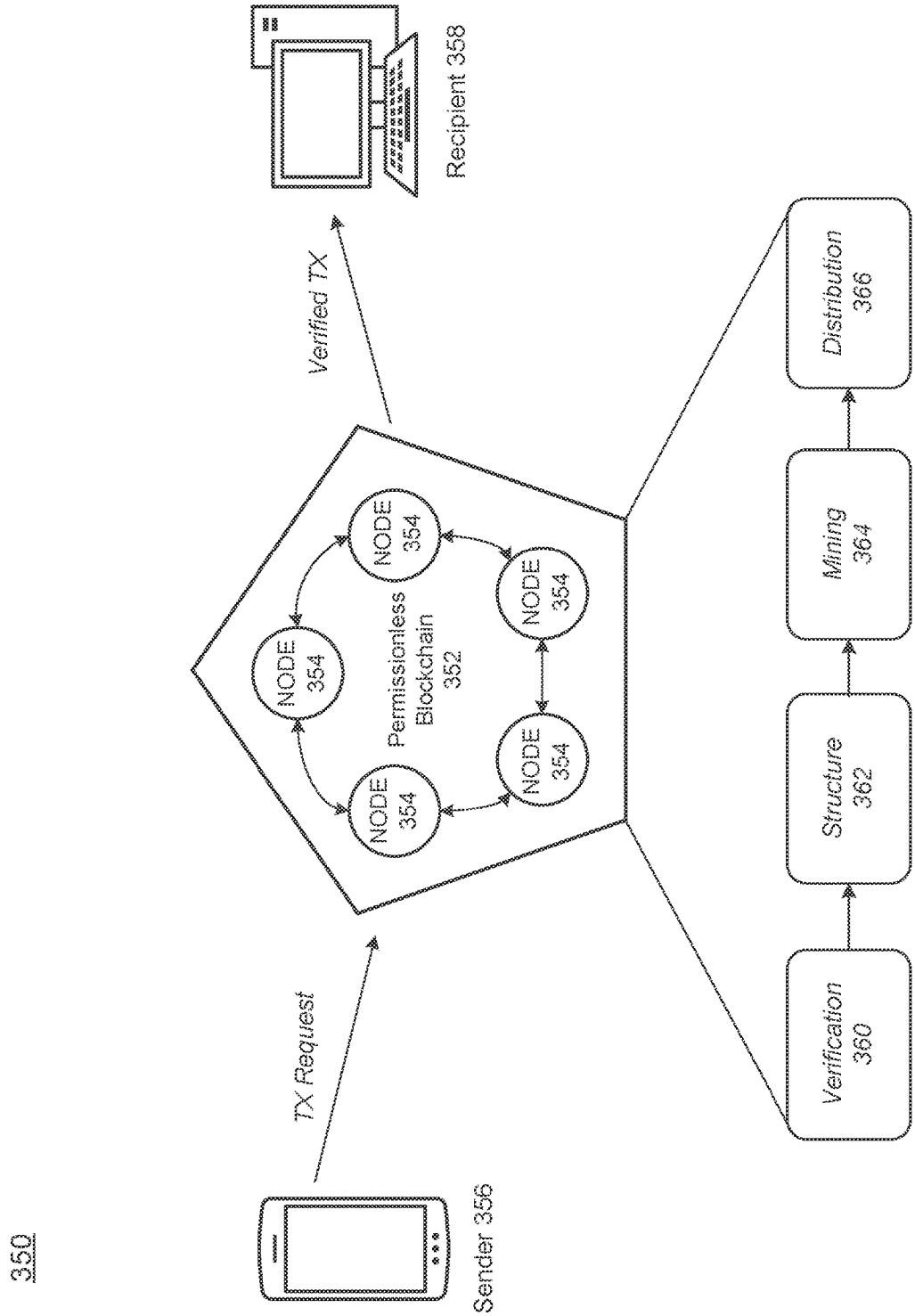

350
Sender 356
TX Request
Recipient 358
Verified TX
NODE 354
NODE 354
NODE 354
NODE 354
NODE 354
Permissionless Blockchain 352
Verification 360
Structure 362
Mining 364
Distribution 366

FIG. 3C

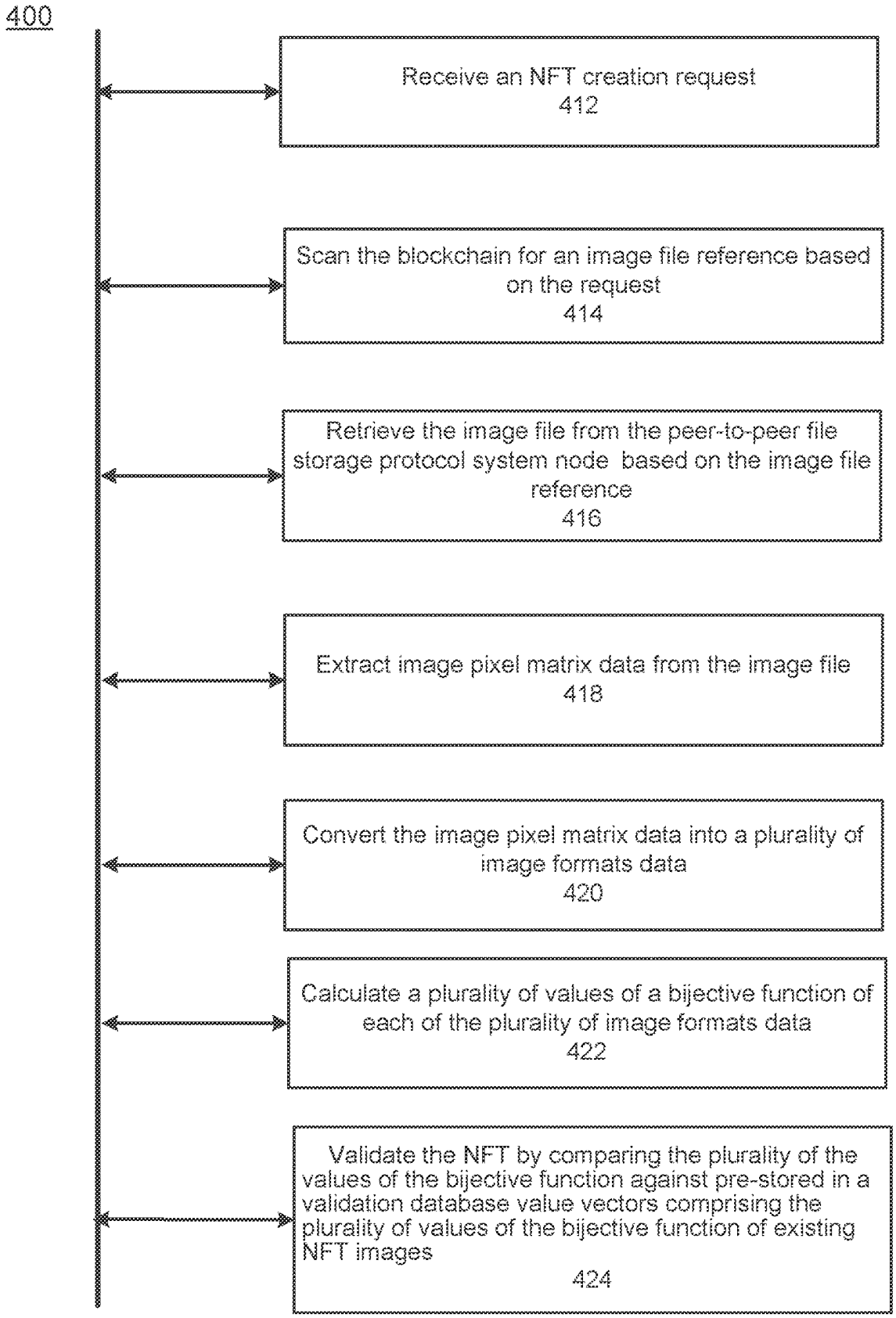

400
Receive an NFT creation request
412
Scan the blockchain for an image file reference based on the request
414
Retrieve the image file from the peer-to-peer file storage protocol system node based on the image file reference
416
Extract image pixel matrix data from the image file
418
Convert the image pixel matrix data into a plurality of image formats data
420
Calculate a plurality of values of a bijective function of each of the plurality of image formats data
422
Validate the NFT by comparing the plurality of the values of the bijective function against pre-stored in a validation database value vectors comprising the plurality of values of the bijective function of existing NFT images
424

FIG. 4

500

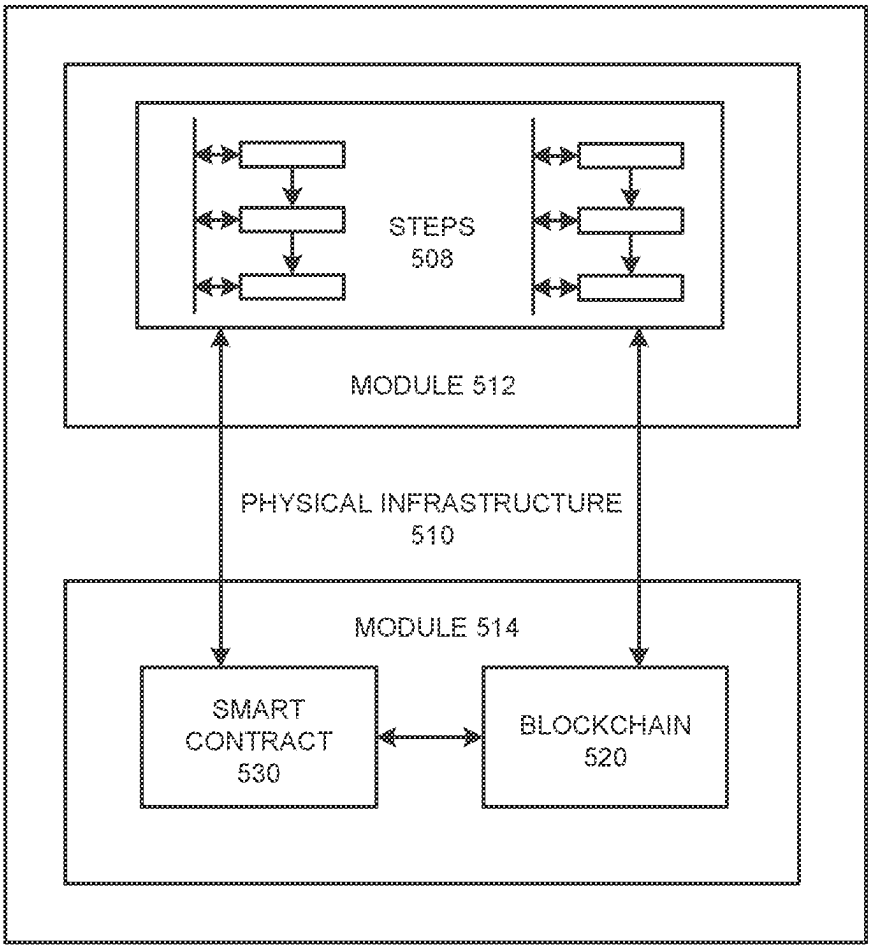

STEPS
508
MODULE 512
PHYSICAL INFRASTRUCTURE
510
MODULE 514
SMART
CONTRACT
530
BLOCKCHAIN
520

FIG. 5A

540

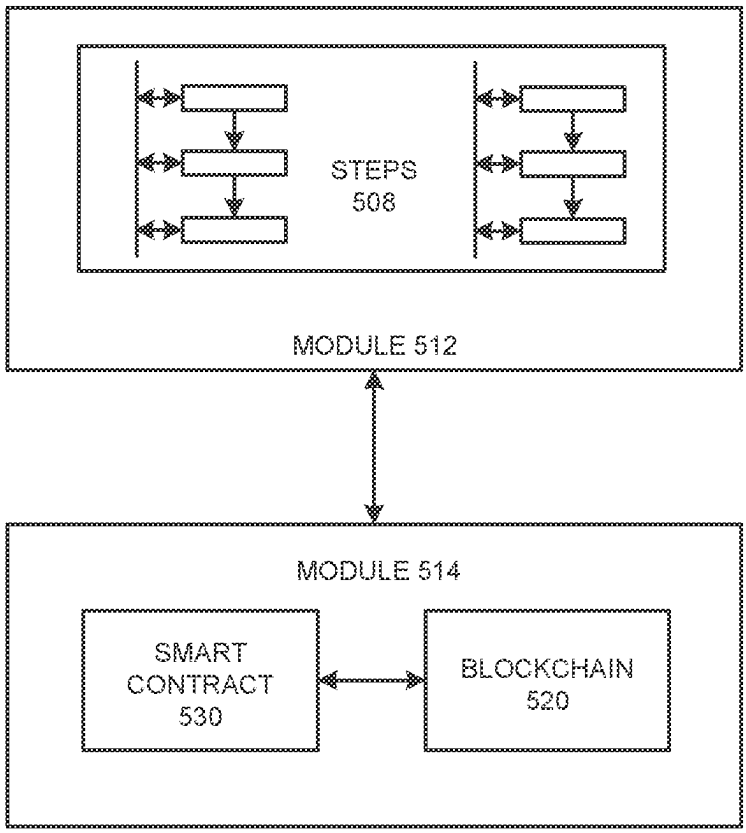

STEPS
508
MODULE 512
MODULE 514
SMART
CONTRACT
530
BLOCKCHAIN
520

FIG. 5B

550

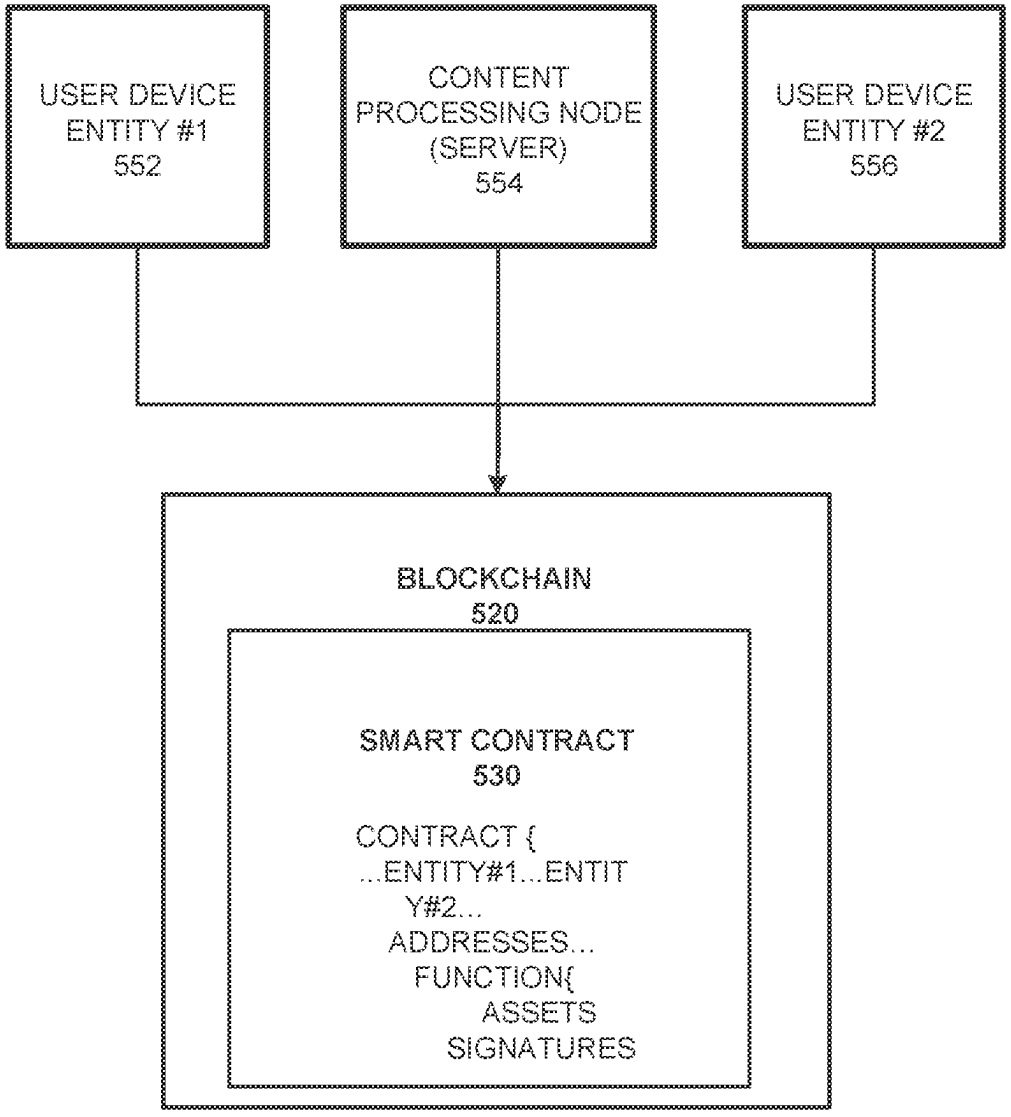

USER DEVICE ENTITY #1 552
CONTENT PROCESSING NODE (SERVER) 554
USER DEVICE ENTITY #2 556
BLOCKCHAIN 520
SMART CONTRACT 530
CONTRACT {
...ENTITY#1...ENTIT
Y#2...
ADDRESSES...
FUNCTION{
ASSETS
SIGNATURES

FIG. 5C

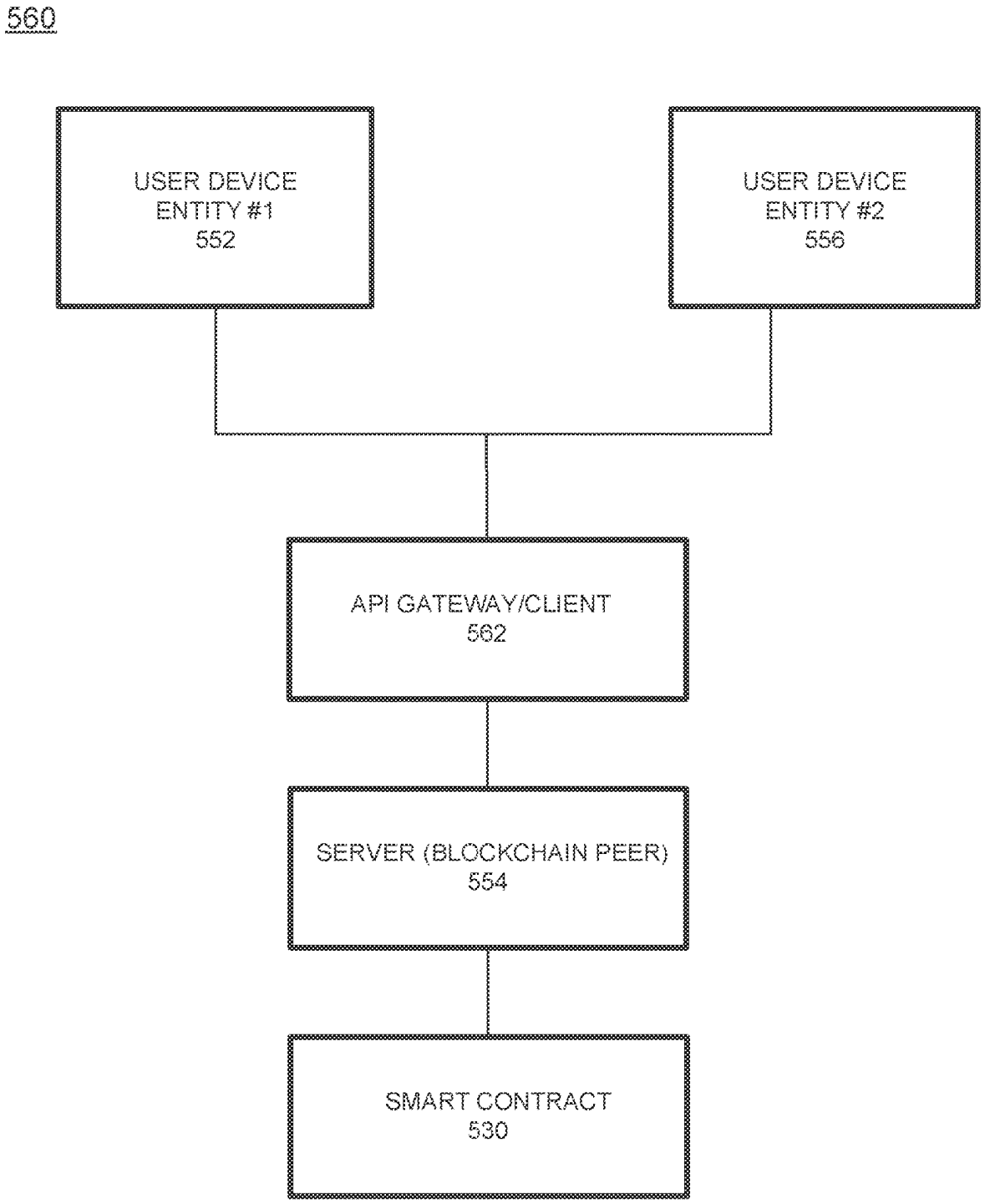

560
USER DEVICE ENTITY #1 552
USER DEVICE ENTITY #2 556
API GATEWAY/CLIENT 562
SERVER (BLOCKCHAIN PEER) 554
SMART CONTRACT 530

FIG. 5D

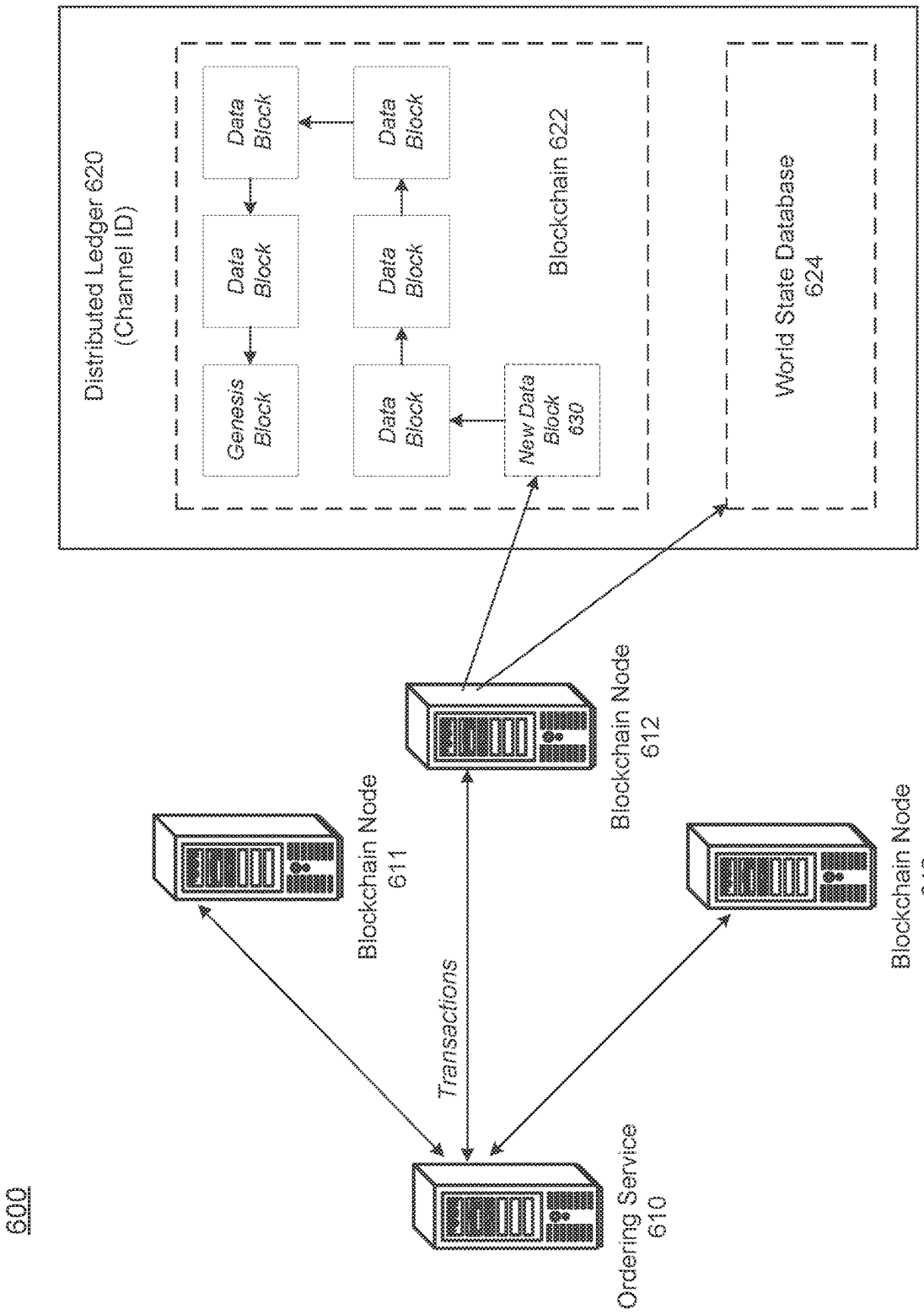

600
Distributed Ledger 620
(Channel ID)
Blockchain 622
Genesis Block
Data Block
Data Block
Data Block
Data Block
Data Block
New Data Block 630
World State Database 624
Blockchain Node 611
Blockchain Node 612
Blockchain Node 613
Transactions
Ordering Service 610

FIG. 6

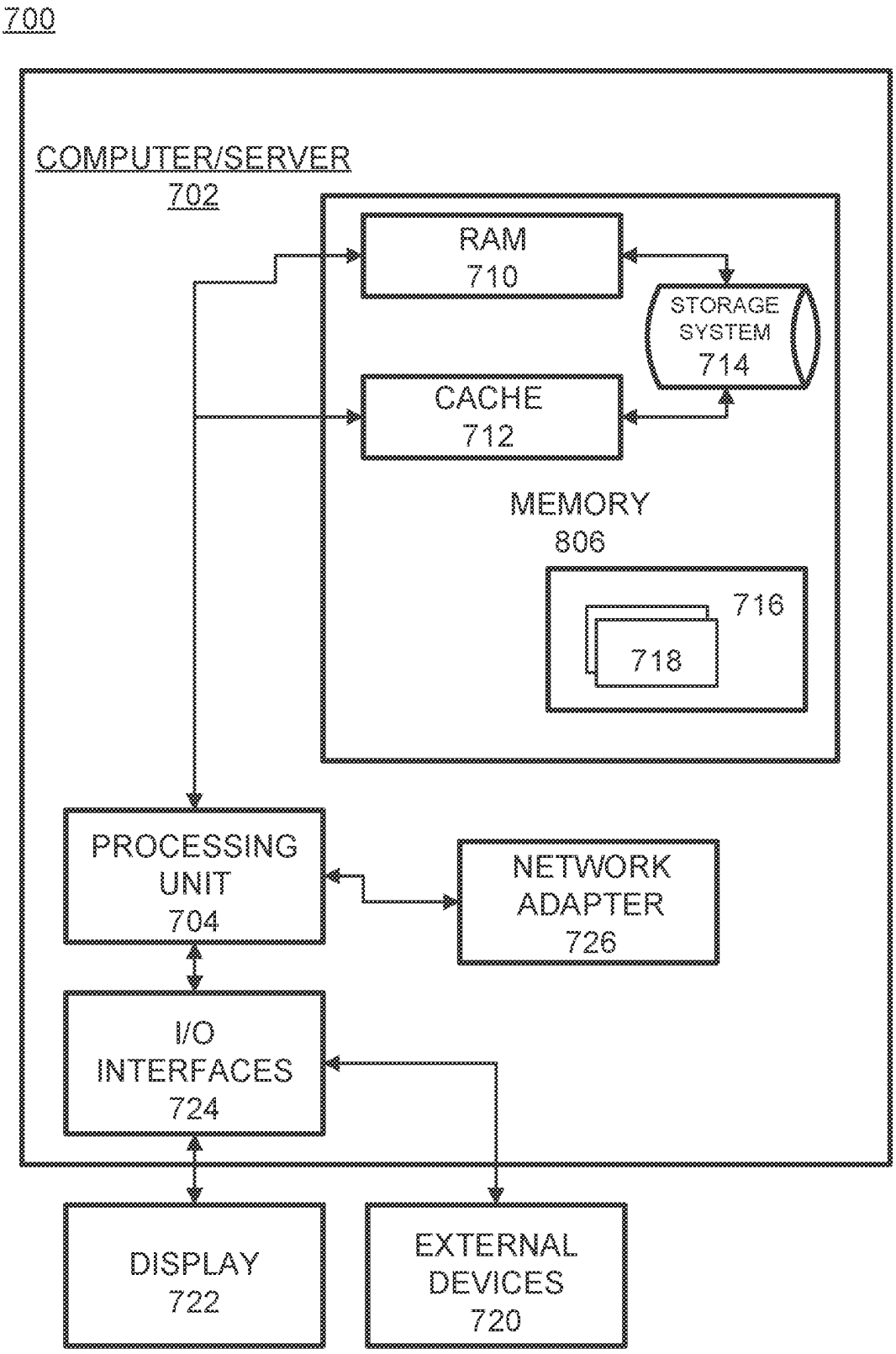

700
COMPUTER/SERVER
702
RAM
710
STORAGE
SYSTEM
714
CACHE
712
MEMORY
806
716
718
PROCESSING
UNIT
704
NETWORK
ADAPTER
726
I/O
INTERFACES
724
DISPLAY
722
EXTERNAL
DEVICES
720

FIG. 7

METHOD AND SYSTEM FOR NFT VALIDATION

BACKGROUND

This application generally relates to image data validation, and more particularly, to validation of NFT assets in blockchain networks.

Currently, blockchain is a decentralized and open system without a central regulation of certification authority. The blockchain technology has been adopted for the trading of artworks as NFT. NFT stands for a "non-fungible token," and it can technically contain anything in a form of digital files such as images, including drawings, animated GIFs, songs, or items in video games. However, no two NFTs are the same. NFTs allow you to buy and sell ownership of unique digital items and keep track of who owns them using the blockchain.

Everyone can create ("mint") a picture as a blockchain asset, without having to prove their ownership. The current authentication mechanisms suggested by the largest NFT blockchain provider Ethereum™ itself and by major trading platforms like OpenSea work only ex-post, to prove ownership after the NFT has been illegally minted and if discovered by the owners. The existing platforms are not able to prevent theft and illegal minting. An identical picture file can be uploaded on Ethereum™ and OpenSea without any obstacles several times.

As such, what is needed is a blockchain-based solution that may be used for recognition of graphic files when uploaded multiple times on the blockchain to prevent the creation and trade of stolen NFT art.

SUMMARY

One example embodiment provides a system that includes a processor and memory, wherein the processor of an NFT processing node is configured to perform one or more of receive an NFT creation request by an NFT processing node connected to a peer-to-peer file storage protocol system node over a blockchain; scan the blockchain for an image file reference based on the request; retrieve the image file from the peer-to-peer file storage protocol system based on the image file reference; extract image pixel matrix data from the image file; convert the image pixel matrix data into a plurality of image formats data; calculating a plurality of values of a bijective or hash function of each of the plurality of image formats data; and validate the NFT by comparing the plurality of the values of the bijective function against pre-stored in a validation database value vectors comprising the plurality of values of the bijective function of existing NFT images. Note that if users try to create an NFT over the disclosed system, the system may receive the image file directly and may have execute a blockchain transaction in the name of the user, using their wallet. Thus, there would be no need to scan the blockchain and the peer-to-peer file storage protocol system to find the image file since the system already has it.

Another example embodiment provides a method that includes one or more of receiving an NFT creation request by an NFT processing node connected to an Inter Planetary File System (IPFS) node over a blockchain; scanning, by the NFT processing node, the blockchain for an image file reference based on the request; retrieving, by the NFT processing node, the image file from the IPFS based on the image file reference; extracting, by the NFT processing node, image pixel matrix data from the image file; converting the image pixel matrix data into a plurality of image formats data; calculating a plurality of values of a bijective function of each of the plurality of image formats data; and validating the NFT by comparing the plurality of the values of the bijective function against pre-stored in a validation database value vectors comprising the plurality of values of the bijective function of existing NFT images.

A further example embodiment provides a non-transitory computer readable medium comprising instructions, that when read by a processor, cause the processor to perform one or more of receiving an NFT creation request by an NFT processing node connected to an Inter Planetary File System (IPFS) node over a blockchain; scanning the blockchain for an image file reference based on the request; retrieving the image file from the IPFS based on the image file reference; extracting image pixel matrix data from the image file; converting the image pixel matrix data into a plurality of image formats data; calculating a plurality of values of a bijective function of each of the plurality of image formats data; and validating the NFT by comparing the plurality of the values of the bijective function against pre-stored in a validation database value vectors comprising the plurality of values of the bijective function of existing NFT images.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. **1**A illustrates a network diagram of a system for level 1 NFT validation and creation of the NFT, according to example embodiments.

FIG. **1**B illustrates a network diagram of a system for level 1 NFT validation and rejection of the NFT, according to example embodiments.

FIG. **1**C illustrates a network diagram of a system for retrieval of information from a blockchain, according to example embodiments.

FIG. **1**D illustrates a network diagram of a system including a detailed description of an NFT processing node, according to example embodiments.

FIG. **2** illustrates an example blockchain architecture configuration, according to example embodiments.

FIG. **3**A illustrates a permissioned network, according to example embodiments.

FIG. **3**B illustrates another permissioned network, according to example embodiments.

FIG. **3**C illustrates a permissionless network, according to example embodiments.

FIG. **4** illustrates a flow diagram of a method of NFT validation, according to example embodiments.

FIG. **5**A illustrates an example system configured to perform one or more operations described herein, according to example embodiments.

FIG. **5**B illustrates another example system configured to perform one or more operations described herein, according to example embodiments.

FIG. **5**C illustrates a further example system configured to utilize a smart contract, according to example embodiments.

FIG. **5**D illustrates yet another example system configured to utilize a blockchain, according to example embodiments.

FIG. **6** illustrates a process for a new block being added to a distributed ledger, according to example embodiments.

FIG. **7** illustrates an example NFT processing node that supports one or more of the example embodiments.

DETAILED DESCRIPTION

It will be readily understood that the instant components, as generally described and illustrated in the figures herein,

may be arranged and designed in a wide variety of different configurations. Thus, the following detailed description of the embodiments of at least one of a method, apparatus, non-transitory computer readable medium and system, as represented in the attached figures, is not intended to limit the scope of the application as claimed but is merely representative of selected embodiments.

The instant features, structures, or characteristics as described throughout this specification may be combined or removed in any suitable manner in one or more embodiments. For example, the usage of the phrases "example embodiments", "some embodiments", or other similar language, throughout this specification refers to the fact that a particular feature, structure, or characteristic described in connection with the embodiment may be included in at least one embodiment. Thus, appearances of the phrases "example embodiments", "in some embodiments", "in other embodiments", or other similar language, throughout this specification do not necessarily all refer to the same group of embodiments, and the described features, structures, or characteristics may be combined or removed in any suitable manner in one or more embodiments. Further, in the diagrams, any connection between elements can permit one-way and/or two-way communication even if the depicted connection is a one-way or two-way arrow. Also, any device depicted in the drawings can be a different device. For example, if a mobile device is shown sending information, a wired device could also be used to send the information.

In addition, while the term "message" may have been used in the description of embodiments, the application may be applied to many types of networks and data. Furthermore, while certain types of connections, messages, and signaling may be depicted in exemplary embodiments, the application is not limited to a certain type of connection, message, and signaling.

Example embodiments provide methods, systems, components, non-transitory computer readable media, devices, and/or networks, which provide for validation of NFTs in blockchain networks.

In one embodiment the application utilizes a decentralized storage (such as a blockchain) that is a distributed storage system, which includes multiple nodes that communicate with each other. The decentralized storage includes an append-only immutable data structure resembling a distributed ledger capable of maintaining records between mutually untrusted parties. The untrusted parties are referred to herein as peers or peer nodes. Each peer maintains a copy of the records and no single peer can modify the records without a consensus being reached among the distributed peers. For example, the peers may execute a consensus protocol to validate blockchain storage and NFT-related transactions, group the storage transactions into blocks, and build a hash chain over the blocks. This process forms the ledger by ordering the storage transactions, as is necessary, for consistency. In various embodiments, a permissioned and/or a permissionless blockchain can be used. In a public or permissionless blockchain, anyone can participate without a specific identity. Public blockchains can involve native cryptocurrency and use consensus based on various protocols such as Proof of Work (PoW). On the other hand, a permissioned blockchain provides secure interactions among a group of entities which share a common goal such as trading NFTs, but which do not fully trust one another.

This application can utilize a blockchain that operates arbitrary, programmable logic, tailored to a decentralized storage scheme and referred to as "smart contracts" or "chaincodes." In some cases, specialized chaincodes may exist for management functions and parameters which are referred to as system chaincode. The application can further utilize smart contracts that are trusted distributed applications which leverage tamper-proof properties of the blockchain database and an underlying agreement between nodes, which is referred to as an endorsement or endorsement policy. Blockchain transactions associated with this application can be "endorsed" before being committed to the blockchain while transactions, which are not endorsed, are disregarded. An endorsement policy allows chaincode to specify endorsers for a transaction in the form of a set of peer nodes that are necessary for endorsement. When a client sends the transaction to the peers specified in the endorsement policy, the transaction is executed to validate the transaction. After validation, the transactions enter an ordering phase in which a consensus protocol is used to produce an ordered sequence of endorsed transactions grouped into blocks.

This application can utilize nodes that are the communication entities of the blockchain system. A "node" may perform a logical function in the sense that multiple nodes of different types can run on the same physical server or on different hosts. Nodes are grouped in trust domains and are associated with logical entities that control them in various ways. Nodes may include different types, such as a client or submitting-client node which submits a transaction-invocation to an endorser (e.g., peer), and broadcasts transaction-proposals to an ordering service (e.g., ordering node). Another type of a node is a peer node which can receive client submitted transactions, commit the transactions and maintain a state and a copy of the ledger of blockchain transactions. Peers can also have the role of an endorser, although it is not a requirement. An ordering-service-node or orderer is a node running the communication service for all nodes, and which implements a delivery guarantee, such as a broadcast to each of the peer nodes in the system when committing transactions and modifying a world state of the blockchain, which is another name for the initial blockchain transaction which normally includes control and setup information.

This application can utilize a ledger that is a sequenced, tamper-resistant record of all state transitions of a blockchain. State transitions may result from chaincode invocations (i.e., transactions) submitted by participating parties (e.g., client nodes, ordering nodes, endorser nodes, peer nodes, etc.). Each participating party (such as a peer node) can maintain a copy of the ledger. A transaction may result in a set of asset key-value pairs being committed to the ledger as one or more operands, such as creates, updates, deletes, and the like. The ledger includes a blockchain which is used to store an immutable, sequenced record in blocks. The ledger also includes a state database which maintains a current state of the blockchain.

This application can utilize a chain that is a transaction log which is structured as hash-linked blocks, and each block contains a sequence of N transactions where N is equal to or greater than one. The block header includes a hash of the block's transactions, as well as a hash of the prior block's header. In this way, all transactions on the ledger may be sequenced and cryptographically linked together. Accordingly, it is not possible to tamper with the ledger data without breaking the hash links. A hash of a most recently added blockchain block represents every transaction on the chain that has come before it, making it possible to ensure that all peer nodes are in a consistent and trusted state. The chain may be stored on a peer node file system (i.e., local, attached

storage, cloud, etc.), efficiently supporting the append-only nature of the blockchain workload.

The current state of the immutable ledger represents the latest values for all keys that are included in the chain transaction log. Since the current state represents the latest key values known to a channel, it is sometimes referred to as a world state. Chaincode invocations execute transactions against the current state data of the ledger. To make these chaincode interactions efficient, the latest values of the keys may be stored in a state database. The state database may be simply an indexed view into the chain's transaction log, it can therefore be regenerated from the chain at any time. The state database may automatically be recovered (or generated if needed) upon peer node startup, and before transactions are accepted.

Some benefits of the instant solutions described and depicted herein include a method and system for provisioning and/or creating of assets such as NFTs in blockchain networks. The exemplary embodiments solve the issues of time and trust by extending features of a database such as immutability, digital signatures and being a single source of truth. The exemplary embodiments provide a solution for making transactions in cryptocurrency and NFTs over a blockchain-based network. The blockchain networks may be homogenous based on the asset type and rules that govern the assets based on the smart contracts.

Blockchain is different from a traditional database in that blockchain is not a central storage, but rather a decentralized, immutable, and secure storage, where nodes must share in changes to records in the storage. Some properties that are inherent in blockchain and which help implement the blockchain include, but are not limited to, an immutable ledger, smart contracts, security, privacy, decentralization, consensus, endorsement, accessibility, and the like, which are further described herein. According to various aspects, the system for buying and selling of NFTs using cryptocurrency in blockchain networks is implemented due to immutable accountability, security, privacy, permitted decentralization, availability of smart contracts, endorsements and accessibility that are inherent and unique to blockchain. In particular, the blockchain ledger data is immutable and that provides for efficient implementation of a method described herein. Also, use of the encryption in the blockchain provides security and builds trust. The smart contract manages the state of the asset to complete the life-cycle. The example blockchains are permission decentralized. Thus, each end user may have its own ledger copy to access. Multiple organizations (and peers) may be on-boarded on the blockchain network. The key organizations (e.g., a NFT processing server) may serve as one of endorsing peers to validate the smart contract execution results, read-set and write-set. In other words, the blockchain inherent features provide for efficient implementation of the system and method described herein.

One of the benefits of the example embodiments is that it improves the functionality of a computing system by implementing a method for rewarding a content recipient and for making NFT-related transactions using blockchain-based systems. Through the blockchain system described herein, a computing system can perform functionality for transferring funds and NFTs in blockchain networks by providing access to capabilities such as distributed ledger, peers, encryption technologies, event handling, etc. Also, the blockchain enables to create a charitable network and make any users or organizations to on-board for participation. As such, the blockchain is not just a database. The blockchain comes with capabilities to create a Business Network (e.g., an NFT exchange network) of users and on-board/off-board organizations to collaborate and execute service processes in the form of smart contracts.

The example embodiments provide numerous benefits over a traditional database. For example, through the blockchain the embodiments provide for immutable accountability, security, privacy, permitted decentralization, availability of smart contracts, endorsements and accessibility that are inherent and unique to the blockchain.

Meanwhile, a traditional database could not be used to implement the example embodiments because it does not bring all parties on the charitable network, it does not create trusted collaboration and does not provide for an efficient storage and transfer of digital assets such as cryptocurrency. The traditional database does not provide for a tamper proof storage and does not provide for preservation of the digital assets being stored. Thus, the proposed method based on blockchain networks cannot be implemented in the traditional database.

Accordingly, the example embodiments provide for a specific solution to a problem in the arts/field of NFTs and asset validation. The example embodiments also change how data may be stored within a block structure of the blockchain. For example, a digital asset data may be securely stored within a certain portion of the data block (i.e., within header, data segment, or metadata). By storing the digital asset data within data blocks of a blockchain, the digital asset data may be appended to an immutable blockchain ledger through a hash-linked chain of blocks. In some embodiments, the data block may be different than a traditional data block by having NFT owner data associated with the digital asset not stored together with the assets within a traditional block structure of a blockchain. By removing the personal owner data associated with the digital asset, the blockchain can provide the benefit of anonymity based on immutable accountability and security.

According to the exemplary embodiments, a method system and computer readable media for validation/authentication of the NFTs are provided.

The exemplary embodiments provide for two levels of NFT validation. The first level provides an authentication of the digital fie using hash functions. Image data from a picture file is used as an input to the function, so that the authentication method is not affected by any changes in file meta data. The function value may be calculated for all available image file formats, so that the authentication method is not affected by saving the image file in another image file format. A vector of function values serves as a unique digital signature.

The second validation level compensates for a weakness of hash functions by adding a second layer of comparison with picture properties to identify if the same hash is caused by collision. The second layer of validation also allows to detect small change in the image (compression, resizing, crop, etc.). The disclosed embodiments provide for a simple control mechanism such as a hash function for prevention of the upload of multiple identical graphic files that may be made to appear as different files by simply resaving the file or saving the image in a different image file format. An image file consists of a pixel matrix data (i.e., the graphics itself) and metadata indicating when was the file saved, source, author, etc. A generalized alternative to hash functions, which avoids the issue of collision, are bijective functions. The bijective functions may also allow the process to run without the second validation step. The bijective functions assign a unique number to an image (i.e., a picture). Every unique picture corresponds to one and only

one unique number. Every number corresponds to only one unique picture. However, due to the large size of this number, this implementation is unlikely to be practical.

Known algorithms may be used to assign a number to a data file, for example MD5 or hash functions. However, these algorithms are not bijective. If these algorithms are applied to the entire image file, the result changes because of changes in the image file metadata. Thus, these algorithms may only suitable for comparing the files at a certain same point in time only. The exemplary embodiments, advantageously, solve this problem by level 1 validation. Only the pixel matrix data is extracted form an image file. The pixel matrix data is fed into a function $f(x_n)=y_n$, where x_n is the n^{th} graphic file and the function $f(x_n)$ is any of:

The hash function MD5;

The hash function SHA-3;

A bijective function $f(x_n)=2x+1$ or any linear function;

A bijective function $f(x_n)=\arctan(x_n)$

The input is a binary cypher consisting of 1s and 0s. However, if the file is saved in another image file format, the numerical representation of the pixel matrix changes. Thus, even a bijective function would return a different result each time. This is addressed by calculating the y_n value for all available image formats. Then, the unique identifier of a unique graphic file number "n" is the vector of values of the bijective functions discussed above for different file formats:

$$\vec{y_n} = \begin{pmatrix} f(.bmp) \\ f(.png) \\ f(.tif) \\ f(.gif) \\ f(.jpeg_{100\%}) \\ f(.jpeg_{95\%}) \\ f(.jpeg_{90\%}) \\ f(.jpeg_{80\%}) \\ f(.heic) \end{pmatrix}$$

As soon as $f(x_n)=y_n$ for a picture match one of the elements of the vector y_n, the picture is identified as the unique picture corresponding to that vector. Since the ".jpeg" format represents a compression of other image file formats, the compression can have variable parameters. Therefore, four entries for ".jpeg" are used according to the level of compression. All unique identifiers for pictures including their owner data created ("minted") through the exemplary platform may be stored in a validation database.

Then, public blockchains and the associated file systems for NFT pictures may be scanned and their unique identifier values may be added to the validation database. This information is public. The values of the unique identifiers are stored in the validation database including data on their NFT owners. It is important to note that hash functions representation contains a limitation named collision: two different files can be represented by the same outcome of the hash function.

However, even with the collision limitation, the level 1 validation allows for detection of duplicate file in the validation database. When a picture is examined, the picture's $\vec{y}$ is calculated and compared against a collection of hash values against a collection of hash values, looking for all cases with the same value. Alternatively, only the entry for the specific file format m can be compared to save processing time. If no such entry is found, the picture is identified as unique.

If corresponding $\vec{y}$ values are found in the validation database, then the graphics with such values will comprise a small set, provided that a reasonably strong hash function method (128 bits) is used. Then, in this small set, a level 2 validation is performed to determine if the reason for the duplication is a collision or if a possibly illegal copy of the picture is detected. To complete the level 1 validation, for each image, statistical image descriptors based on colors, textures, local features and gradients in a vector form $g(x_n)=\vec{h}$ are stored in the validation database in case additional validation is needed.

Then, the statistical image descriptors of the analyzed graphic x_n may be calculated and compared with the statistical image descriptors of the images with matching y_n values in the level 1 validation. If there is one match in the vector of statistical image descriptors, the images are identified as identical. If there are no matches at all, then the image is identified as a collision. In case of the collision, the $\vec{y}$ entry is duplicated in the validation database and the $\vec{h}$ value is stored in the validation database.

FIG. **1**A illustrates a network diagram of a system for level 1 NFT validation and creation of the NFT, according to example embodiments. Referring to FIG. **1**A, the example network includes an NFT processing node **102** connected to other nodes such as a web server **106** over blockchain network (not shown). The NFT processing node **102** may include additional components and that some of the components described herein may be removed and/or modified without departing from a scope of the NFT processing node **102** disclosed herein. The NFT processing node **102** may be a computing device or a server computer, or the like, and may include a processor **104**, which may be a semiconductor-based microprocessor, a central processing unit and/or another hardware device. Although a single processor **104** is depicted, it should be understood that the NFT processing node **102** may include multiple processors, multiple cores, or the like, without departing from the scope of the NFT processing node **102**.

The NFT processing node **102** may also include a non-transitory computer readable medium **112** that may have stored thereon machine-readable instructions executable by the processor **104**. Examples of the machine-readable instructions are shown as **114-121** and are further discussed below. Examples of the non-transitory computer readable medium **112** may include an electronic, magnetic, optical, or other physical storage device that contains or stores executable instructions. For example, the non-transitory computer readable medium **112** may be a Random-Access Memory (RAM), an Electrically Erasable Programmable Read-Only Memory (EEPROM), a hard disk, an optical disc, or other type of storage device.

A participant node **101** sends an image file **111** to a web server **106**. The web server **106** generates a request for NFT creation **113** and sends it to the NFT processing node **102**. A processor **104** may fetch, decode, and execute the machine-readable instructions **114** to retrieve the image file from the request. The processor **104** may fetch, decode, and execute the machine-readable instructions **116** to extract image pixel matrix data from the image file. The processor **104** may fetch, decode, and execute the machine-readable instructions **118** to calculate $f(x_n)$ for an image file format m. As discussed above, a vector value y_n value is generated for all available image formats. Then, the unique identifier of a unique graphic file number "n" is the vector of values of the bijective function for different file formats:

$$\vec{y_n} = \begin{pmatrix} f(.bmp) \\ f(.png) \\ f(.tif) \\ f(.gif) \\ f(.jpeg_{100\%}) \\ f(.jpeg_{95\%}) \\ f(.jpeg_{90\%}) \\ f(.jpeg_{80\%}) \\ f(.heic) \end{pmatrix}$$

At step **118**, as soon as $f(x_n)=y_n$ for an image match of one of the elements of the vector y_n of image **126***a* retrieved from a vector database **125** (also referred to as a validation database herein), the image is identified as the unique image corresponding to that vector y_n. The processor **104** may fetch, decode, and execute the machine-readable instructions **119** to move to the level 2 validation. Otherwise, the processor **104** may fetch, decode, and execute the machine-readable instructions **120** to save the image file in all available formats discussed above. The processor **104** may fetch, decode, and execute the machine-readable instructions **121** to calculate the value of the $f(x_n)$, save the results into a new vector y_n of the image **126***b* on the vector database **125** and generate NFT creation allowance message **123** that is sent to the web server **106** and forwarded back to the participant node **101**.

FIG. **1**B illustrates a network diagram of a system for level 1 NFT validation and rejection of the NFT, according to example embodiments. Referring to FIG. **1**B, the example network includes an NFT processing node **102** connected to other nodes such as a web server **106** over blockchain network (not shown). The NFT processing node **102** may include additional components and that some of the components described herein may be removed and/or modified without departing from a scope of the NFT processing node **102** disclosed herein. The NFT processing node **102** may be a computing device or a server computer, or the like, and may include a processor **104**, which may be a semiconductor-based microprocessor, a central processing unit and/or another hardware device. Although a single processor **104** is depicted, it should be understood that the NFT processing node **102** may include multiple processors, multiple cores, or the like, without departing from the scope of the NFT processing node **102**.

The NFT processing node **102** may also include a non-transitory computer readable medium **112** that may have stored thereon machine-readable instructions executable by the processor **104**. Examples of the machine-readable instructions are shown as **114-130** and are further discussed below. Examples of the non-transitory computer readable medium **112** may include an electronic, magnetic, optical, or other physical storage device that contains or stores executable instructions. For example, the non-transitory computer readable medium **112** may be a Random-Access Memory (RAM), an Electrically Erasable Programmable Read-Only Memory (EEPROM), a hard disk, an optical disc, or other type of storage device.

A participant node **101** sends an image file **111** to a web server **106**. The web server **106** generates a request for NFT creation **113** and sends it to the NFT processing node **102**. A processor **104** may fetch, decode, and execute the machine-readable instructions **114** to retrieve the image file from the request. The processor **104** may fetch, decode, and execute the machine-readable instructions **116** to extract image pixel matrix data from the image file. The processor **104** may fetch, decode, and execute the machine-readable instructions **117** to calculate $f(x_n)$ for an image file format m. As discussed above, a vector value y_n value is generated for all available image formats. Then, the unique identifier of a unique graphic file number "n" is the vector of values of the bijective function (or a hash function) for different file formats:

$$\vec{y_n} = \begin{pmatrix} f(.bmp) \\ f(.png) \\ f(.tif) \\ f(.gif) \\ f(.jpeg_{100\%}) \\ f(.jpeg_{95\%}) \\ f(.jpeg_{90\%}) \\ f(.jpeg_{80\%}) \\ f(.heic) \end{pmatrix}$$

At step **118**, as soon as $f(x_n)=y_n$ for an image match of one of the elements of the vector y_n retrieved from a vector database **125** (also referred to as a validation database herein), the image is identified as the unique image corresponding to that vector y_n. The processor **104** may fetch, decode, and execute the machine-readable instructions **121** to end the level 1 validation if $f(x_n)$ does not equal to y_n. The processor **104** may fetch, decode, and execute the machine-readable instructions **122** to calculate a value of $g(x_n)$ for a set of q matches on level 1 validation based on vector h retrieved from the vector database **127**. If, at step **125**, $g(x_n)$ equals to at lest one element g(xg) in the vector database **127**, the processor **104** may fetch, decode, and execute the machine-readable instructions **130** to add a collision notice to the vector database **128**, duplicate $f(x_n)$ value and add $g(x_n)$ value. Otherwise, the processor **104** may fetch, decode, and execute the machine-readable instructions **127** to reject NFT creation and instructions **129** to send a notification to the owner of the $f(x_n)$ value in the vector database **128**. The NFT processing node **102** may send a message **132** on rejected NFT creation to the web server **106** which forwards the message to the participant node **101**.

FIG. **1**C illustrates a network diagram of a system for retrieval of information from a blockchain, according to example embodiments.

Referring to FIG. **1**C, the example network includes an NFT processing node **102** connected to other nodes such as a web server **106** over blockchain network (not shown). The NFT processing node **102** may include additional components and that some of the components described herein may be removed and/or modified without departing from a scope of the NFT processing node **102** disclosed herein. The NFT processing node **102** may be a computing device or a server computer, or the like, and may include a processor **104**, which may be a semiconductor-based microprocessor, a central processing unit (CPU), an application specific integrated circuit (ASIC), a field-programmable gate array (FPGA), and/or another hardware device. Although a single processor **104** is depicted, it should be understood that the NFT processing node **102** may include multiple processors, multiple cores, or the like, without departing from the scope of the NFT processing node **102**.

The NFT processing node **102** may also include a non-transitory computer readable medium **112** that may have stored thereon machine-readable instructions executable by the processor **104**. Examples of the machine-readable instructions are shown as **114-130** and are further discussed below. Examples of the non-transitory computer readable medium **112** may include an electronic, magnetic, optical, or

other physical storage device that contains or stores executable instructions. For example, the non-transitory computer readable medium **112** may be a Random-Access Memory (RAM), an Electrically Erasable Programmable Read-Only Memory (EEPROM), a hard disk, an optical disc, or other type of storage device.

A participant node **101** sends a request for NFT creation **113** to a web server **106**. The web server **106** forwards the request for NFT creation **113** to the NFT processing node **102**. A processor **104** may fetch, decode, and execute the machine-readable instructions **133**. In order to have a sufficient number of values for vectors $\vec{y}$ and $\vec{h}$ in the validation database **127** and **128** (see FIG. **1**B), pictures (images) already minted on the blockchain are analyzed.

Usually, the image files are not stored directly on the blockchain, but on a peer-to-peer file storage protocol, which in the case of Ethereum is called Inter Planetary File System (IPFS) protocol. Other instances include permaweb and its instance arewave for Solana™. Thus, if the blockchain transaction points to an IP on the IPFS, the picture (i.e., image) file is retrieved from there. In general, it is easier to retrieve information form the blockchain from a peer node computer which participates in the consensus group. However, in exchange, participants in the consensus group have to offer computing power. Therefore, three variations of the exemplary embodiments may be used to account for variable account of pictures to be verified:

- a low number of pictures for verification: the server which runs the algorithm and retrieves the data for pictures created elsewhere on the blockchain is not a part of the consensus group;
- a high number of pictures for verification: the server which runs the algorithm and retrieves the data for pictures created elsewhere on the blockchain is a part of the consensus group;
- a flexible mode: if there is a verification workload higher than a pre-determined threshold, the server which runs the algorithm and retrieves the data for pictures created elsewhere on the blockchain enters the consensus group. If the verification workload falls below the pre-determined threshold, the server which runs the algorithm and retrieves the data for pictures created elsewhere on the blockchain leaves the consensus group.

Accordingly, if a number of total requests exceed a threshold p at step **133**, the processor **104** may execute the instructions **134** to retrieve data from the blockchain and the IPFS after joining the consensus group. Otherwise, the processor **104** may execute the instructions **135** to continue in regular data retrieval mode. For images retrieved from the blockchain or from the IPFS, $\vec{y}$ and $\vec{h}$ may be calculated and their values may be saved in the validation database.

FIG. **1**D illustrates a network diagram of a system including a detailed description of an NFT processing node, according to example embodiments.

Referring to FIG. **1**D, the example network includes an NFT processing node **102** connected to other nodes such as IPFS node **140** over blockchain **142**. The NFT processing node **102** may include additional components and that some of the components described herein may be removed and/or modified without departing from a scope of the NFT processing node **102** disclosed herein. The NFT processing node **102** may be a computing device or a server computer, or the like, and may include a processor **104**, which may be a semiconductor-based microprocessor, a central processing unit (CPU), an application specific integrated circuit (ASIC), a field-programmable gate array (FPGA), and/or another hardware device. Although a single processor **104** is depicted, it should be understood that the NFT processing node **102** may include multiple processors, multiple cores, or the like, without departing from the scope of the NFT processing node **102**.

The NFT processing node **102** may also include a non-transitory computer readable medium **112** that may have stored thereon machine-readable instructions executable by the processor **104**. Examples of the machine-readable instructions are shown as **144**-**154** and are further discussed below. Examples of the non-transitory computer readable medium **112** may include an electronic, magnetic, optical, or other physical storage device that contains or stores executable instructions. For example, the non-transitory computer readable medium **112** may be a Random-Access Memory (RAM), an Electrically Erasable Programmable Read-Only Memory (EEPROM), a hard disk, an optical disc, or other type of storage device.

The peer-to-peer file storage protocol system node **140** host an image file **111** associated with the NFT. The NFT processing node **102** scans the blockchain **142** for NFT reference **143** to the image file **111** that needs to be validated. A processor **104** may fetch, decode, and execute the machine-readable instructions **144** to retrieve the image file from the peer-to-peer file storage protocol system node according to the blockchain reference **143**. The processor **104** may fetch, decode, and execute the machine-readable instructions **116** to extract image pixel matrix data from the image file **111**. The processor **104** may fetch, decode, and execute the machine-readable instructions **148** to convert the image pixel matrix data into all available image formats. The processor **104** may fetch, decode, and execute the machine-readable instructions **150** to calculate $f(x)$ for all available image file formats of the image file **111**. As discussed above, a vector value y_n value may be generated for all available image formats. Then, the unique identifier of a unique graphic file number "n" is the vector of values of the bijective function for different file formats:

$$\vec{y_n} = \begin{pmatrix} f(.bmp) \\ f(.png) \\ f(.tif) \\ f(.gif) \\ f(.jpeg_{100\%}) \\ f(.jpeg_{95\%}) \\ f(.jpeg_{90\%}) \\ f(.jpeg_{80\%}) \\ f(.heic) \end{pmatrix}$$

Accordingly, the processor **104** may fetch, decode, and execute the machine-readable instructions **151** to generate vector y_n. The processor **104** may fetch, decode, and execute the machine-readable instructions **152** to save vector y_n in the vector database **127** referred herein as a validation database. Then, the processor **104** may fetch, decode, and execute the machine-readable instructions **153** to calculate a value of $g(x_n)$ discussed in more details below. The processor **104** may fetch, decode, and execute the machine-readable instructions **154** to save vector h_n in the vector database **127** discussed above with reference to FIG. **1**B.

As discussed above, the system and method, according to the disclosed embodiments, provide for a minting protection of the NFTs. When someone tries to mint a new picture (image) on the disclosed platform, the system compares its unique identifier $f(x_n)=y_n$ with the vectors in the verification

database. If the system identifies matches one of the elements of an existing vector in the verification database, the level 2 validation is started. If the image(s) are found to be identical also in the level 2 validation, then the minting of the picture is rejected.

If an artist would like to create a series of pictures, then the system marks in the verification database that there are "p" multiple entries for the same picture. The creation of multiple identical NFTs is allowed up to this threshold value "p". Transfer and listing of pictures minted elsewhere is implemented as follows. When someone offers a picture for sale on platform website, the system calculates its unique identifier y_n. If the unique identifier is already in the validation database, the level 2 validation is initiated. If the validation indicates an identical picture, the seller can proceed with the sale if they are identified as the last owner in the validation database. If the validation database contains another person as the last owner, the seller must provide a proof of having purchased the file from the last owner.

The proof can be a chain of transactions on a public blockchain (assuming that there has been a mistake in the validation database) or a physical purchase contract. In both cases, an examination and update of the validation database will be performed. This is to account for the higher risk of fraud.

If the level 2 validation indicates a collision, the system allows the customer to sell the picture on the platform's website and adds the new y_n for the collision item to the validation database. If the unique identifier is not in the validation database, and the mint has occurred after the last scan of the blockchain and or IPFS, the platform allows the customer to sell the picture on platform's website and add the new y_n for the collision item to the validation database. The system checks the chain of transactions on the blockchain automatically and verifies that the seller is the last owner according to the chain of transactions. The system allows the sale and adds the unique identifier to the validation database. If the mint has occurred before the last scan, a manual investigation is launched. The sale is not allowed until the investigation is finished.

If the submitted file is in an image file format of a high compression, like ".jpeg" of a compression level of 80%, and it is not found to match any $\vec{y}_n$ in the validation database, the reconstructed $\vec{y}_n$ is not used as a definitive signature of the image file, since the original non-compressed files may have been different than the platform reconstructions. In this case, other techniques based on statistical similarity may be used for the validation.

FIG. **2** illustrates a blockchain architecture configuration **200**, according to example embodiments. Referring to FIG. **2**A, the blockchain architecture **200** may include certain blockchain elements, for example, a group of blockchain nodes **202**. The blockchain nodes **202** may include one or more nodes **204-210** (these four nodes are depicted by example only). These nodes participate in a number of activities, such as blockchain transaction addition and validation process (consensus). One or more of the blockchain nodes **204-210** may endorse transactions based on endorsement policy and may provide an ordering service for all blockchain nodes in the architecture **200**. A blockchain node may initiate a blockchain authentication and seek to write to a blockchain immutable ledger stored in blockchain layer **216**, a copy of which may also be stored on the underpinning physical infrastructure **214**. The blockchain configuration may include one or more applications **224** which are linked to application programming interfaces (APIs) **222** to access and execute stored program/application code **220** (e.g., chaincode, smart contracts, etc.) which can be created according to a customized configuration sought by participants and can maintain their own state, control their own assets, and receive external information. This can be deployed as a transaction and installed, via appending to the distributed ledger, on all blockchain nodes **204-210**.

The blockchain base or platform **212** may include various layers of blockchain data, services (e.g., cryptographic trust services, virtual execution environment, etc.), and underpinning physical computer infrastructure that may be used to receive and store new transactions and provide access to auditors which are seeking to access data entries. The blockchain layer **216** may expose an interface that provides access to the virtual execution environment necessary to process the program code and engage the physical infrastructure **214**. Cryptographic trust services **218** may be used to verify transactions such as asset exchange transactions and keep information private.

The blockchain architecture configuration of FIG. **2**A may process and execute program/application code **220** via one or more interfaces exposed, and services provided, by blockchain platform **212**. The code **220** may control blockchain assets such as NFTs. For example, the code **220** can store and transfer data, and may be executed by nodes **204-210** in the form of a smart contract and associated chaincode with conditions or other code elements subject to its execution. As a non-limiting example, smart contracts may be created to execute reminders, updates, and/or other notifications subject to the changes, updates, etc. The smart contracts can themselves be used to identify rules associated with authorization and access requirements and usage of the ledger. For example, the information **226** representing the NFT may be processed by one or more processing entities (e.g., virtual machines) included in the blockchain layer **216**. The result **228** may include the NFT being transferred to a user. The physical infrastructure **214** may be utilized to retrieve any of the data or information described herein.

A smart contract may be created via a high-level application and programming language, and then written to a block in the blockchain. The smart contract may include executable code which is registered, stored, and/or replicated with a blockchain (e.g., distributed network of blockchain peers). A transaction is an execution of the smart contract code which can be performed in response to conditions associated with the smart contract being satisfied. The executing of the smart contract may trigger a trusted modification(s) to a state of a digital blockchain ledger. The modification(s) to the blockchain ledger caused by the smart contract execution may be automatically replicated throughout the distributed network of blockchain peers through one or more consensus protocols.

The smart contract may write data to the blockchain in the format of key-value pairs. Furthermore, the smart contract code can read the values stored in the blockchain and use them in application operations. The smart contract code can write the output of various logic operations into the blockchain. The code may be used to create a temporary data structure in a virtual machine or other computing platform. Data written to the blockchain can be public and/or can be encrypted and maintained as private. The temporary data that is used/generated by the smart contract is held in memory by the supplied execution environment, then deleted once the data needed for the blockchain is identified.

A chaincode may include the code interpretation of a smart contract, with additional features. As described herein, the chaincode may be program code deployed on a com-

puting network, where it is executed and validated by chain validators together during a consensus process. The chaincode receives a hash and retrieves from the blockchain a hash associated with the data template created by use of a previously stored feature extractor. If the hashes of the hash identifier and the hash created from the stored identifier template data match, then the chaincode sends an authorization key to the requested service. The chaincode may write to the blockchain data associated with the cryptographic details.

FIG. **3**A illustrates an example of a permissioned blockchain network **300**, which features a distributed, decentralized peer-to-peer architecture. In this example, a blockchain user **302** may initiate a transaction to the permissioned blockchain **304**. In this example, the transaction can be a deploy, invoke, or query, and may be issued through a client-side application leveraging an SDK, directly through an API, etc. Networks may provide access to a regulator **306**, such as an auditor. A blockchain network operator **308** manages member permissions, such as enrolling the regulator **306** as an "auditor" and the blockchain user **302** as a "client." An auditor could be restricted only to querying the ledger whereas a client could be authorized to deploy, invoke, and query certain types of chaincode.

A blockchain developer **310** can write chaincode and client-side applications. The blockchain developer **310** can deploy chaincode directly to the network through an interface. To include credentials from a traditional data source **312** in chaincode, the developer **310** could use an out-of-band connection to access the data. In this example, the blockchain user **302** connects to the permissioned blockchain **304** through a peer node **314**. Before proceeding with any transactions, the peer node **314** retrieves the user's enrollment and transaction certificates from a certificate authority **316**, which manages user roles and permissions. In some cases, blockchain users must possess these digital certificates in order to transact on the permissioned blockchain **304**. Meanwhile, a user attempting to utilize chaincode may be required to verify their credentials on the traditional data source **312**. To confirm the user's authorization, chaincode can use an out-of-band connection to this data through a traditional processing platform **318**.

FIG. **3**B illustrates another example of a permissioned blockchain network **320**, which features a distributed, decentralized peer-to-peer architecture. In this example, a blockchain user **322** may submit a transaction to the permissioned blockchain **324**. In this example, the transaction can be deploy, invoke, or query, and may be issued through a client-side application leveraging an SDK, directly through an API, etc. Networks may provide access to a regulator **326**, such as an auditor. A blockchain network operator **328** manages member permissions, such as enrolling the regulator **326** as an "auditor" and the blockchain user **322** as a "client." An auditor could be restricted only to querying the ledger whereas a client could be authorized to deploy, invoke, and query certain types of chaincode.

A blockchain developer **330** writes chaincode and client-side applications. The blockchain developer **330** can deploy chaincode directly to the network through an interface. To include credentials from a traditional data source **332** in chaincode, the developer **330** could use an out-of-band connection to access the data. In this example, the blockchain user **322** connects to the network through a peer node **334**. Before proceeding with any transactions, the peer node **334** retrieves the user's enrollment and transaction certificates from the certificate authority **336**. In some cases, blockchain users must possess these digital certificates in order to transact on the permissioned blockchain **324**. Meanwhile, a user attempting to utilize chaincode may be required to verify their credentials on the traditional data source **332**. To confirm the user's authorization, chaincode can use an out-of-band connection to this data through a traditional processing platform **338**.

In some embodiments, the blockchain herein may be a permissionless blockchain. In contrast with permissioned blockchains which require permission to join, anyone can join a permissionless blockchain. For example, to join a permissionless blockchain a user may create a personal address and begin interacting with the network, by submitting transactions, and hence adding entries to the ledger. Additionally, all parties have the choice of running a node on the system and employing the mining protocols to help verify transactions.

FIG. **3**C illustrates a process **350** of a transaction being processed by a permissionless blockchain **352** including a plurality of nodes **354**. A sender **356** desires to send payment or some other form of value (e.g., NFTs or any other assets that can be encapsulated in a digital record) to a recipient **358** via the permissionless blockchain **352**. In one embodiment, each of the sender device **356** and the recipient device **358** may have digital wallets (associated with the blockchain **352**) that provide user interface controls and a display of transaction parameters. In response, the transaction is broadcast throughout the blockchain **352** to the nodes **354**. Depending on the blockchain's **352** network parameters the nodes verify **360** the transaction based on rules (which may be pre-defined or dynamically allocated) established by the permissionless blockchain **352** creators. For example, this may include verifying identities of the parties involved (e.g., NFT owner, etc.). The transaction may be verified immediately or it may be placed in a queue with other transactions and the nodes **354** determine if the transactions are valid based on a set of network rules.

In structure **362**, valid transactions are formed into a block and sealed with a lock (hash). This process may be performed by mining nodes among the nodes **354**. Mining nodes may utilize additional software specifically for mining and creating blocks for the permissionless blockchain **352**. Each block may be identified by a hash (e.g., 256-bit number, etc.) created using an algorithm agreed upon by the network. Each block may include a header, a pointer or reference to a hash of a previous block's header in the chain, and a group of valid transactions. The reference to the previous block's hash is associated with the creation of the secure independent chain of blocks.

Before blocks can be added to the blockchain, the blocks must be validated. Validation for the permissionless blockchain **352** may include a proof-of-work (PoW) which is a solution to a puzzle derived from the block's header. Although not shown in the example of FIG. **3**C, another process for validating a block is proof-of-stake. Unlike the proof-of-work, where the algorithm rewards miners who solve mathematical problems, with the proof of stake, a creator of a new block is chosen in a deterministic way, depending on its wealth, also defined as "stake." Then, a similar proof is performed by the selected/chosen node.

With mining **364**, nodes try to solve the block by making incremental changes to one variable until the solution satisfies a network-wide target. This creates the PoW thereby ensuring correct answers. In other words, a potential solution must prove that computing resources were drained in solving the problem. In some types of permissionless blockchains, miners may be rewarded with value (e.g., coins, etc.) for correctly mining a block.

Here, the PoW process, alongside the chaining of blocks, makes modifications of the blockchain extremely difficult, as an attacker must modify all subsequent blocks in order for the modifications of one block to be accepted. Furthermore, as new blocks are mined, the difficulty of modifying block increases, and the number of subsequent blocks increases. With distribution **366**, the successfully validated block is distributed through the permissionless blockchain **352** and all nodes **354** add the block to a majority chain which is the permissionless blockchain's **352** auditable ledger. Furthermore, the value in the transaction submitted by the sender **356** is deposited or otherwise transferred to the digital wallet of the recipient device **358**.

FIG. **4** illustrates a flow diagram **400** of an example method of NFT validation, according to example embodiments. Referring to FIG. **4**, the method **400** may include one or more of the steps described below.

FIG. **4** illustrates a flow chart of an example method executed by the NFT processing node **102** (see FIG. **1**D). It should be understood that method **400** depicted in FIG. **4** may include additional operations and that some of the operations described therein may be removed and/or modified without departing from the scope of the method **400**. The description of the method **400** is also made with reference to the features depicted in FIG. **1**D for purposes of illustration. Particularly, the processor **104** of the NFT processing node **102** may execute some or all of the operations included in the method **400**.

With reference to FIG. **4**, at block **412**, the processor **104** may receive an NFT creation request. At block **414**, the processor **104** may scan a blockchain for an image file reference based on the request. At block **416**, the processor **104** may retrieve the image file from the peer-to-peer file storage protocol system node based on the image file reference. At block **418**, the processor **104** may extract image pixel matrix data from the image file. At block **420**, the processor **104** may convert the image pixel matrix data into a plurality of image formats data. At block **422**, the processor **104** may calculate a plurality of values of a bijective function of each of the plurality of image formats data. At block **424**, the processor **104** may validate the NFT by comparing the plurality of the values of the bijective function against pre-stored in a validation database value vectors comprising the plurality of values of the bijective function of existing NFT images.

FIG. **5**A illustrates an example system **500** that includes a physical infrastructure **510** configured to perform various operations according to example embodiments. Referring to FIG. **5**A, the physical infrastructure **510** includes a module **512** and a module **514**. The module **514** includes a blockchain **520** and a smart contract **530** (which may reside on the blockchain **520**), that may execute any of the operational steps **508** (in module **512**) included in any of the example embodiments. The steps/operations **508** may include one or more of the embodiments described or depicted and may represent output or written information that is written or read from one or more smart contracts **530** and/or blockchains **520**. The physical infrastructure **510**, the module **512**, and the module **514** may include one or more computers, servers, processors, memories, and/or wireless communication devices. Further, the module **512** and the module **514** may be a same module.

FIG. **5**B illustrates another example system **540** configured to perform various operations according to example embodiments. Referring to FIG. **5**B, the system **540** includes a module **512** and a module **514**. The module **514** includes a blockchain **520** and a smart contract **530** (which may reside on the blockchain **520**), that may execute any of the operational steps **508** (in module **512**) included in any of the example embodiments. The steps/operations **508** may include one or more of the embodiments described or depicted and may represent output or written information that is written or read from one or more smart contracts **530** and/or blockchains **520**. The physical infrastructure **510**, the module **512**, and the module **514** may include one or more computers, servers, processors, memories, and/or wireless communication devices. Further, the module **512** and the module **514** may be a same module.

FIG. **5**C illustrates an example system configured to utilize a smart contract configuration among contracting parties and a mediating server configured to enforce the smart contract terms on the blockchain according to example embodiments. Referring to FIG. **5**C, the configuration **550** may represent a communication session, an asset (e.g., NFT) transfer session or a process or procedure that is driven by a smart contract **530** which explicitly identifies one or more user devices **552** and/or **556**. The execution, operations and results of the smart contract execution may be managed by the NFT processing node (e.g., a server) **554**. Content of the smart contract **530** may require digital signatures by one or more of the entities **552** and **556** which are parties to the smart contract transaction. The results of the smart contract execution may be written to a blockchain **520** as a blockchain transaction. The smart contract **530** resides on the blockchain **520** which may reside on one or more computers, servers, processors, memories, and/or wireless communication devices.

FIG. **5**D illustrates a system **560** including a blockchain, according to example embodiments. Referring to the example of FIG. **5**D, an application programming interface (API) gateway **562** provides a common interface for accessing blockchain logic (e.g., smart contract **530** or other chaincode) and data (e.g., distributed ledger, etc.). In this example, the API gateway **562** is a common interface for performing transactions (invoke, queries, etc.) on the blockchain by connecting one or more entities **552** and **556** to a blockchain peer (i.e., server **554**). Here, the server **554** is a blockchain network peer component that holds a copy of the world state and a distributed ledger allowing clients **552** and **556** to query data on the world state as well as submit transactions into the blockchain network where, depending on the smart contract **530** and endorsement policy, endorsing peers will run the smart contracts **530**.

The above embodiments may be implemented in hardware, in a computer program executed by a processor, in firmware, or in a combination of the above. A computer program may be embodied on a computer readable medium, such as a storage medium. For example, a computer program may reside in random access memory ("RAM"), flash memory, read-only memory ("ROM"), erasable programmable read-only memory ("EPROM"), electrically erasable programmable read-only memory ("EEPROM"), registers, hard disk, a removable disk, a compact disk read-only memory ("CD-ROM"), or any other form of storage medium known in the art.

An exemplary storage medium may be coupled to the processor such that the processor may read information from, and write information to, the storage medium. In the alternative, the storage medium may be integral to the processor. The processor and the storage medium may reside in an application specific integrated circuit ("ASIC"). In the alternative, the processor and the storage medium may reside as discrete components.

FIG. **6** illustrates a process **600** of a new block being added to a distributed ledger **620**, according to example embodiments. Referring to FIG. **6**, clients (not shown) may submit transactions to blockchain nodes **611**, **612**, and/or **613**. Clients may be instructions received from any source to enact activity on the blockchain **620**. As an example, the clients may be applications that act on behalf of a requester, such as a device, person or entity to propose transactions for the blockchain. The plurality of blockchain peers (e.g., blockchain nodes **611**, **612**, and **613**) may maintain a state of the blockchain network and a copy of the distributed ledger **620**. Different types of blockchain nodes/peers may be present in the blockchain network including endorsing peers which simulate and endorse transactions proposed by clients and committing peers which verify endorsements, validate transactions, and commit transactions to the distributed ledger **620**. In this example, the blockchain nodes **611**, **612**, and **613** may perform the role of endorser node, committer node, or both.

The distributed ledger **620** includes a blockchain which stores immutable, sequenced records in blocks, and a state database **624** (current world state) maintaining a current state of the blockchain **622**. One distributed ledger **620** may exist per channel and each peer maintains its own copy of the distributed ledger **620** for each channel of which they are a member. The blockchain **622** is a transaction log, structured as hash-linked blocks where each block contains a sequence of N transactions. The linking of the blocks (shown by arrows) may be generated by adding a hash of a prior block's header within a block header of a current block. In this way, all transactions on the blockchain **622** are sequenced and cryptographically linked together preventing tampering with blockchain data without breaking the hash links. Furthermore, because of the links, the latest block in the blockchain **622** represents every transaction that has come before it. The blockchain **622** may be stored on a peer file system (local or attached storage), which supports an append-only blockchain workload.

The current state of the blockchain **622** and the distributed ledger **620** may be stored in the state database **624**. Here, the current state data represents the latest values for all keys ever included in the chain transaction log of the blockchain **622**. Chaincode invocations execute transactions against the current state in the state database **624**. To make these chaincode interactions extremely efficient, the latest values of all keys are stored in the state database **624**. The state database **624** may include an indexed view into the transaction log of the blockchain **622**, it can therefore be regenerated from the chain at any time. The state database **624** may automatically get recovered (or generated if needed) upon peer startup, before transactions are accepted.

Endorsing nodes receive transactions from clients and endorse the transaction based on simulated results. Endorsing nodes hold smart contracts which simulate the transaction proposals. When an endorsing node endorses a transaction, the endorsing node creates a transaction endorsement which is a signed response from the endorsing node to the client application indicating the endorsement of the simulated transaction. The method of endorsing a transaction depends on an endorsement policy which may be specified within chaincode. An example of an endorsement policy is "the majority of endorsing peers must endorse the transaction." Different channels may have different endorsement policies. Endorsed transactions are forward by the client application to ordering service **610**.

The ordering service **610** accepts endorsed transactions, orders them into a block, and delivers the blocks to the committing peers. For example, the ordering service **610** may initiate a new block when a threshold of transactions has been reached, a timer times out, or another condition. In the example depicted in FIG. **6**, blockchain node **612** is a committing peer that has received a new data new data block **630** for storage on blockchain **620**. The first block in the blockchain may be referred to as a genesis block which includes information about the blockchain, its members, the data stored therein, etc.

The ordering service **610** may be made up of a cluster of orderers. The ordering service **610** does not process transactions, smart contracts, or maintain the shared ledger. Rather, the ordering service **610** may accept the endorsed transactions and specifies the order in which those transactions are committed to the distributed ledger **620**. The architecture of the blockchain network may be designed such that the specific implementation of 'ordering' (e.g., Solo, Kafka, BFT, etc.) becomes a pluggable component.

Transactions are written to the distributed ledger **620** in a consistent order. The order of transactions is established to ensure that the updates to the state database **624** are valid when they are committed to the network. Unlike a cryptocurrency blockchain system (e.g., Bitcoin, etc.) where ordering occurs through the solving of a cryptographic puzzle, or mining, in this example the parties of the distributed ledger **620** may choose the ordering mechanism that best suits that network.

When the ordering service **610** initializes a new data block **630**, the new data block **630** may be broadcast to committing peers (e.g., blockchain nodes **611**, **612**, and **613**). In response, each committing peer validates the transaction within the new data block **630** by checking to make sure that the read set and the write set still match the current world state in the state database **624**. Specifically, the committing peer can determine whether the read data that existed when the endorsers simulated the transaction is identical to the current world state in the state database **624**. When the committing peer validates the transaction, the transaction is written to the blockchain **622** on the distributed ledger **620**, and the state database **624** is updated with the write data from the read-write set. If a transaction fails, that is, if the committing peer finds that the read-write set does not match the current world state in the state database **624**, the transaction ordered into a block will still be included in that block, but it will be marked as invalid, and the state database **624** will not be updated.

FIG. **7** illustrates an example NFT processing node **700** that supports one or more of the example embodiments described and/or depicted herein. The NFT processing node **700** comprises a computer system/server **702**, which is operational with numerous other general purpose or special purpose computing system environments or configurations. Examples of well-known computing systems, environments, and/or configurations that may be suitable for use with computer system/server **702** include, but are not limited to, personal computer systems, server computer systems, thin clients, thick clients, hand-held or laptop devices, multiprocessor systems, microprocessor-based systems, set top boxes, programmable consumer electronics, network PCs, minicomputer systems, mainframe computer systems, and distributed cloud computing environments that include any of the above systems or devices, and the like.

The computer system/server **702** may be described in the general context of computer system-executable instructions, such as program modules, being executed by a computer system. Generally, program modules may include routines, programs, objects, components, logic, data structures, and so

on that perform particular tasks or implement particular abstract data types. Computer system/server **702** may be practiced in distributed cloud computing environments where tasks are performed by remote processing devices that are linked through a communications network. In a distributed cloud computing environment, program modules may be located in both local and remote computer system storage media including memory storage devices.

As shown in FIG. **7**, computer system/server **702** in the NFT processing node **700** is shown in the form of a general-purpose computing device. The components of computer system/server **702** may include, but are not limited to, one or more processors or processing units **704**, a system memory **706**, and a bus that couples various system components including system memory **706** to processor **704**.

The bus represents one or more of any of several types of bus structures, including a memory bus or memory controller, a peripheral bus, an accelerated graphics port, and a processor or local bus using any of a variety of bus architectures. By way of example, and not limitation, such architectures include Industry Standard Architecture (ISA) bus, Micro Channel Architecture (MCA) bus, Enhanced ISA (EISA) bus, Video Electronics Standards Association (VESA) local bus, and Peripheral Component Interconnects (PCI) bus.

Computer system/server **702** typically includes a variety of computer system readable media. Such media may be any available media that is accessible by computer system/server **702**, and it includes both volatile and non-volatile media, removable and non-removable media. System memory **706**, in one embodiment, implements the flow diagrams of the other figures. The system memory **706** can include computer system readable media in the form of volatile memory, such as random-access memory (RAM) **710** and/or cache memory **712**. Computer system/server **702** may further include other removable/non-removable, volatile/non-volatile computer system storage media. By way of example only, storage system **714** can be provided for reading from and writing to a non-removable, non-volatile magnetic media (not shown and typically called a "hard drive"). Although not shown, a magnetic disk drive for reading from and writing to a removable, non-volatile magnetic disk, and an optical disk drive for reading from or writing to a removable, non-volatile optical disk such as a CD-ROM, DVD-ROM or other optical media can be provided. In such instances, each can be connected to the bus by one or more data media interfaces. As will be further depicted and described below, memory **706** may include at least one program product having a set (e.g., at least one) of program modules that are configured to carry out the functions of various embodiments of the application.

Program/utility **716**, having a set (at least one) of program modules **718**, may be stored in memory **706** by way of example, and not limitation, as well as an operating system, one or more application programs, other program modules, and program data. Each of the operating system, one or more application programs, other program modules, and program data or some combination thereof, may include an implementation of a networking environment. Program modules **718** generally carry out the functions and/or methodologies of various embodiments of the application as described herein.

As will be appreciated by one skilled in the art, aspects of the present application may be embodied as a system, method, or computer program product. Accordingly, aspects of the present application may take the form of an entirely hardware embodiment, an entirely software embodiment (including firmware, resident software, micro-code, etc.) or an embodiment combining software and hardware aspects that may all generally be referred to herein as a "circuit," "module" or "system." Furthermore, aspects of the present application may take the form of a computer program product embodied in one or more computer readable medium(s) having computer readable program code embodied thereon.

Computer system/server **702** may also communicate with one or more external devices **720** such as a keyboard, a pointing device, a display **722**, etc.; one or more devices that enable a user to interact with computer system/server **702**; and/or any devices (e.g., network card, modem, etc.) that enable computer system/server **702** to communicate with one or more other computing devices. Such communication can occur via I/O interfaces **724**. Still yet, computer system/server **702** can communicate with one or more networks such as a local area network (LAN), a general wide area network (WAN), and/or a public network (e.g., the Internet) via network adapter **726**. As depicted, network adapter **726** communicates with the other components of computer system/server **702** via a bus. It should be understood that although not shown, other hardware and/or software components could be used in conjunction with computer system/server **702**. Examples, include, but are not limited to: microcode, device drivers, redundant processing units, external disk drive arrays, RAID systems, tape drives, and data archival storage systems, etc.

Although an exemplary embodiment of at least one of a system, method, and non-transitory computer readable medium has been illustrated in the accompanied drawings and described in the foregoing detailed description, it will be understood that the application is not limited to the embodiments disclosed, but is capable of numerous rearrangements, modifications, and substitutions as set forth and defined by the following claims. For example, the capabilities of the system of the various figures can be performed by one or more of the modules or components described herein or in a distributed architecture and may include a transmitter, receiver or pair of both. For example, all or part of the functionality performed by the individual modules, may be performed by one or more of these modules. Further, the functionality described herein may be performed at various times and in relation to various events, internal or external to the modules or components. Also, the information sent between various modules can be sent between the modules via at least one of: a data network, the Internet, a voice network, an Internet Protocol network, a wireless device, a wired device and/or via plurality of protocols. Also, the messages sent or received by any of the modules may be sent or received directly and/or via one or more of the other modules.

One skilled in the art will appreciate that a "system" could be embodied as a personal computer, a server, a console, a personal digital assistant (PDA), a cell phone, a tablet computing device, a smartphone or any other suitable computing device, or combination of devices. Presenting the above-described functions as being performed by a "system" is not intended to limit the scope of the present application in any way but is intended to provide one example of many embodiments. Indeed, methods, systems and apparatuses disclosed herein may be implemented in localized and distributed forms consistent with computing technology.

It should be noted that some of the system features described in this specification have been presented as modules, in order to more particularly emphasize their implementation independence. For example, a module may be

implemented as a hardware circuit comprising custom very large-scale integration (VLSI) circuits or gate arrays, off-the-shelf semiconductors such as logic chips, transistors, or other discrete components. A module may also be implemented in programmable hardware devices such as field programmable gate arrays, programmable array logic, programmable logic devices, graphics processing units, or the like.

A module may also be at least partially implemented in software for execution by various types of processors. An identified unit of executable code may, for instance, comprise one or more physical or logical blocks of computer instructions that may, for instance, be organized as an object, procedure, or function. Nevertheless, the executables of an identified module need not be physically located together but may comprise disparate instructions stored in different locations which, when joined logically together, comprise the module and achieve the stated purpose for the module. Further, modules may be stored on a computer-readable medium, which may be, for instance, a hard disk drive, flash device, random access memory (RAM), tape, or any other such medium used to store data.

Indeed, a module of executable code could be a single instruction, or many instructions, and may even be distributed over several different code segments, among different programs, and across several memory devices. Similarly, operational data may be identified and illustrated herein within modules and may be embodied in any suitable form and organized within any suitable type of data structure. The operational data may be collected as a single data set or may be distributed over different locations including over different storage devices, and may exist, at least partially, merely as electronic signals on a system or network.

It will be readily understood that the components of the application, as generally described and illustrated in the figures herein, may be arranged and designed in a wide variety of different configurations. Thus, the detailed description of the embodiments is not intended to limit the scope of the application as claimed but is merely representative of selected embodiments of the application.

One having ordinary skill in the art will readily understand that the above may be practiced with steps in a different order, and/or with hardware elements in configurations that are different than those which are disclosed. Therefore, although the application has been described based upon these preferred embodiments, it would be apparent to those of skill in the art that certain modifications, variations, and alternative constructions would be apparent.

While preferred embodiments of the present application have been described, it is to be understood that the embodiments described are illustrative only and the scope of the application is to be defined solely by the appended claims when considered with a full range of equivalents and modifications (e.g., protocols, hardware devices, software platforms etc.) thereto.

What is claimed is:

1. A system for validation of non-fungible tokens (NFTs), comprising:

a processor of an NFT processing node connected to a peer-to-peer file storage protocol system node over a blockchain;

a memory on which are stored machine-readable instructions that when executed by the processor, cause the processor to:

receive an NFT creation request;

scan the blockchain for an image file reference based on the request;

retrieve the image file from the peer-to-peer file storage protocol system based on the image file reference;

extract image pixel matrix data from the image file;

convert the image pixel matrix data into a plurality of image formats data;

calculate a plurality of values of a bijective function of each of the plurality of image formats data;

validate an NFT associated with the NFT creation request by comparing the plurality of the values of the bijective function against pre-stored in a validation database value vectors comprising the plurality of values of the bijective function of existing NFT images; and

responsive to no match of each of the plurality of the values of the bijective function to the pre-stored value vectors, generate a new value vector based on the plurality of values of the bijective function and store the new value vector in the validation database,

wherein the instructions further cause the processor to generate a value vector y based on the plurality of values of the bijective function of the image matrix data based on the plurality of the image file formats, wherein the y_n comprising:

$$\overrightarrow{y_n} = \begin{pmatrix} f(.bmp) \\ f(.png) \\ f(.tif) \\ f(.gif) \\ f(.jpeg_{100\%}) \\ f(.jpeg_{95\%}) \\ f(.jpeg_{90\%}) \\ f(.jpeg_{80\%}) \\ f(.heic) \end{pmatrix}.$$

2. The system of claim **1**, wherein the instructions further cause the processor to, responsive to a collision representing a match of at least one of the plurality of the values of the bijective function to at least one value of the pre-stored value vectors, compare statistical image pixel matrix data against a pre-stored statistical image descriptors vector of a matched stored NFT image defined by the pre-stored value vector.

3. The system of claim **2**, wherein the statistical image descriptors vector comprises a plurality of descriptors based on colors, textures and gradients of the matched stored NFT image.

4. The system of claim **2**, wherein the instructions further cause the processor to calculate statistical descriptors of the image pixel matrix data to be compared against the pre-stored statistical image descriptors vector of the matched stored NFT image.

5. The system of claim **4**, wherein the instructions further cause the processor to, responsive to no matching of the statistical descriptors of the image pixel matrix data to at least one feature of the pre-stored statistical image descriptors vector, generate a duplicate entry of a value vector corresponding to the image pixel matrix data in the validation database along with a statistical descriptors vector of the image pixel matrix data.

6. The system of claim **1**, wherein the instructions further cause the processor to scan public blockchains and associated file systems for NFT pictures, generate unique value vectors for images associated with the NFT pictures and add the value vectors to the validation database.

7. A method for validation of non-fungible tokens (NFTs), comprising:

receiving an NFT creation request by an NFT processing node connected to a peer-to-peer file storage protocol system node over a blockchain;

scanning, by the NFT processing node, the blockchain for an image file reference based on the request;

retrieving, by the NFT processing node, the image file from the peer-to-peer file storage protocol system based on the image file reference;

extracting, by the NFT processing node, image pixel matrix data from the image file;

converting, by the NFT processing node, the image pixel matrix data into a plurality of image formats data;

calculating, by the NFT processing node, a plurality of values of a bijective function of each of the plurality of image formats data;

validating an NFT associated with the NFT creation request by comparing the plurality of the values of the bijective function against pre-stored in a validation database value vectors comprising the plurality of values of the bijective function of existing NFT images; and

responsive to no match of each of the plurality of the values of the bijective function to the pre-stored value vectors, generating a new value vector based on the plurality of values of the bijective and store the new value vector in the validation database; and

generating a value vector y_n based on the plurality of values of the bijective function of the image matrix data based on the plurality of the image file formats, wherein the y_n comprising:

$$\vec{y_n} = \begin{pmatrix} f(.bmp) \\ f(.png) \\ f(.tif) \\ f(.gif) \\ f(.jpeg_{100\%}) \\ f(.jpeg_{95\%}) \\ f(.jpeg_{90\%}) \\ f(.jpeg_{80\%}) \\ f(.heic) \end{pmatrix}.$$

8. The method of claim **7**, further comprising, responsive to a collision representing a match of at least one of the plurality of the values of the bijective function to at least one value of the pre-stored value vectors, comparing statistical image pixel matrix data against a pre-stored statistical image descriptors vector of a matched stored NFT image defined by the pre-stored value vector.

9. The method of claim **7**, further comprising calculating statistical descriptors of the image pixel matrix data to be compared against the pre-stored statistical image descriptors vector of the matched stored NFT image.

10. The method of claim **9**, further comprising, responsive to no matching of the statistical descriptors of the image pixel matrix data to at least one feature of the pre-stored statistical image descriptors vector, generating a duplicate entry of a value vector corresponding to the image pixel matrix data in the validation database along with a statistical descriptors vector of the image pixel matrix data.

11. The method of claim **7**, further comprising scanning public blockchains and associated file systems for NFT pictures, generating unique value vectors for images associated with the NFT pictures and adding the value vectors to the validation database.

12. A non-transitory computer readable medium comprising instructions, that when read by a processor, cause the processor to perform:

receiving an NFT creation request;

scanning a blockchain for an image file reference based on the request;

retrieving the image file from an Inter Planetary File System based on the image file reference;

extracting image pixel matrix data from the image file;

converting the image pixel matrix data into a plurality of image formats data;

calculating a plurality of values of a bijective function of each of the plurality of image formats data;

validating an NFT associated with the NFT creation request by comparing the plurality of the values of the bijective function against pre-stored in a validation database value vectors comprising the plurality of values of the bijective function of existing NFT images; and

responsive to no match of each of the plurality of the values of the bijective function to the pre-stored value vectors, generating a new value vector based on the plurality of values of the bijective and storing the new value vector in the validation database;

generating a value vector y_n based on the plurality of values of the bijective function of the image matrix data based on the plurality of the image file formats, wherein the y_n comprising:

$$\vec{y_n} = \begin{pmatrix} f(.bmp) \\ f(.png) \\ f(.tif) \\ f(.gif) \\ f(.jpeg_{100\%}) \\ f(.jpeg_{95\%}) \\ f(.jpeg_{90\%}) \\ f(.jpeg_{80\%}) \\ f(.heic) \end{pmatrix}.$$

13. The non-transitory computer readable medium of claim **12**, further comprising instructions, that when read by the processor, cause the processor to generate a value vector y_n based on the plurality of values of the bijective function of the image matrix data based on the plurality of the image file formats, wherein the y_n comprising:

$$\vec{y_n} = \begin{pmatrix} f(.bmp) \\ f(.png) \\ f(.tif) \\ f(.gif) \\ f(.jpeg_{100\%}) \\ f(.jpeg_{95\%}) \\ f(.jpeg_{90\%}) \\ f(.jpeg_{80\%}) \\ f(.heic) \end{pmatrix}.$$

14. The non-transitory computer readable medium of claim **12**, further comprising instructions, that when read by the processor, cause the processor to, responsive to a collision representing a match of at least one of the plurality of the values of the bijective function to at least one value of the pre-stored value vectors, compare statistical image pixel matrix data against a pre-stored statistical image descriptors vector of a matched stored NFT image defined by the pre-stored value vector.

15. The non-transitory computer readable medium of claim 14, further comprising instructions, that when read by the processor, cause the processor to, responsive to no matching of generated statistical descriptors of the image pixel matrix data to at least one feature of the pre-stored statistical image descriptors vector, generate a duplicate entry of a value vector corresponding to the image pixel matrix data in the validation database along with a statistical descriptors vector of the image pixel matrix data.

16. The non-transitory computer readable medium of claim 12, further comprising instructions, that when read by the processor, cause the processor to scan public blockchains and associated file systems for NFT pictures, generate unique value vectors for images associated with the NFT pictures and add the value vectors to the validation database.

* * * * *